



Lurnexa Publications

**DBMS
Solved Query Bank**

11 Case Studies

Case Study 1: Sailors-Boats-Reserves

ER Model, Relational Model, Constraints — Units I-II

Setup — Schema & Data

Create Tables:

```
CREATE TABLE Sailors(sid INT PRIMARY KEY, sname VARCHAR(40) NOT
NULL,
    rating INT CHECK(rating>=1 AND rating<=10), age DECIMAL(4,1));
```

```
CREATE TABLE Boats(bid INT PRIMARY KEY, bname VARCHAR(40) NOT
NULL,
    color VARCHAR(20), boat_type VARCHAR(20));
```

```
CREATE TABLE Reserves(sid INT, bid INT, rdate DATE NOT NULL,
    PRIMARY KEY(sid,bid,rdate),
    FOREIGN KEY(sid) REFERENCES Sailors(sid),
    FOREIGN KEY(bid) REFERENCES Boats(bid));
```

Insert Values:

```
INSERT INTO Sailors VALUES
(1,'Alice',8,45.0),(2,'Bob',10,63.5),(3,'Charlie',5,35.0),
(4,'Diana',7,42.0),(5,'Eve',9,51.0),(6,'Frank',6,38.5);
```

```
INSERT INTO Boats VALUES
(101,'Nautilus','Red','Sailboat'),(102,'Phoenix','Green','Yacht'),
(103,'Titan','Blue','Speedboat'),(104,'Emerald','Green','Sailboat');
```

```
INSERT INTO Reserves VALUES
(1,101,'2026-01-15'),(1,102,'2026-02-01'),(2,101,'2026-01-20'),
(2,102,'2026-03-10'),(3,104,'2026-02-15'),(4,101,'2026-03-05'),
(5,103,'2026-01-30'),(6,104,'2026-02-20');
```

Sailors Table:

SID	SNAME	RATING	AGE
1	Alice	8	45
2	Bob	10	63.5
3	Charlie	5	35
4	Diana	7	42
5	Eve	9	51

SID	SNAME	RATING	AGE
6	Frank	6	38.5

Boats Table:

BID	BNAME	COLOR	BOAT_TYPE
101	Nautilus	Red	Sailboat
102	Phoenix	Green	Yacht
103	Titan	Blue	Speedboat
104	Emerald	Green	Sailboat

Reserves Table:

SID	BID	RDATE
1	101	2026-01-15
1	102	2026-02-01
2	101	2026-01-20
2	102	2026-03-10
3	104	2026-02-15
4	101	2026-03-05
5	103	2026-01-30
6	104	2026-02-20

Q1 ER Model — Entities, Attributes, Primary Keys**Query:**

```
SELECT 'Sailors: sid|sname|rating|age; Boats: bid|bname|color|boat_type;
Reserves: sid,bid,rdate (relationship)' AS ER_structure;
```

Output:

ER_STRUCTURE
Sailors: sid sname rating age; Boats: bid bname color boat_type; Reserves: sid,bid,rdate (relationship)

Solution: ER model identifies entities Sailors and Boats connected by relationship Reserves.

Q2 Primary Key, Candidate Keys, Foreign Key Constraints

Query:

```
SELECT 'PK(Sailors)=sid, PK(Boats)=bid, PK(Reserves)=(sid,bid,rdate),
      FK(sid,bid) refs Sailors,Boats' AS keys;
```

Output:

KEYS
PK(Sailors)=sid, PK(Boats)=bid, PK(Reserves)=(sid,bid,rdate), FK(sid,bid) refs Sailors,Boats

Solution: Composite PK for Reserves; FKs ensure referential integrity.

Q3 Domain Constraints — Sailors with Rating Between 1 and 10

Query:

```
SELECT sid, sname, rating
FROM Sailors
WHERE rating>=1 AND rating<=10
ORDER BY rating DESC;
```

Output:

SID	SNAME	RATING
2	Bob	10
5	Eve	9
1	Alice	8
4	Diana	7
6	Frank	6
3	Charlie	5

Solution: CHECK constraint enforces domain: rating must be 1-10.

Q4 Uniqueness Constraint — Duplicate Primary Key Attempt

Query:

```
INSERT INTO Sailors VALUES (1,'Duplicate',7,50.0);
```

Output:

ERROR
PK constraint violation: sid=1 already exists

Solution: PK uniqueness enforced by DBMS; duplicate rejected.

Q5 Referential Integrity — Insert Reservation with Non-Existent Boat

Query:

```
INSERT INTO Reserves VALUES (1,999,'2026-04-01');
```

Output:

ERROR
FK constraint violated: boat_id 999 not in Boats

Solution: Foreign key ensures only valid boat references allowed.

Q6 Schema Design — ER to Relational Schema (Normalization)

Query:

```
SELECT 'Sailors(sid,sname,rating,age),  
Boats(bid,bname,color,boat_type),  
Reserves(sid,bid,rdate)' AS normalized_schema;
```

Output:

NORMALIZED_SCHEMA
Sailors(sid,sname,rating,age), Boats(bid,bname,color,boat_type), Reserves(sid,bid,rdate)

Solution: Relational schema avoids redundancy; 1NF compliant.

Q7 Complex Aggregation with Skill-Level Classification

Query:

```
WITH sailor_profile AS (  
  SELECT s.sid, s.sname, s.rating,  
         COUNT(DISTINCT r.bid) AS boats_reserved,  
         COUNT(DISTINCT r.rdate) AS reservations,  
         GROUP_CONCAT(DISTINCT b.color) AS boat_colors  
  FROM Sailors s  
  LEFT JOIN Reserves r ON s.sid=r.sid  
  LEFT JOIN Boats b ON r.bid=b.bid  
  GROUP BY s.sid, s.sname, s.rating  
)  
SELECT sid, sname, rating, boats_reserved, reservations,  
       CASE WHEN rating>=8 THEN 'Expert'
```

```

    WHEN rating >= 6 THEN 'Intermediate'
    ELSE 'Novice' END AS skill_level
FROM sailor_profile
ORDER BY boats_reserved DESC;

```

Output:

SID	SNAME	RATING	BOATS_RESERVED	RESERVATIONS	SKILL_LEVEL
2	Bob	10	2	2	Expert
1	Alice	8	2	2	Expert
5	Eve	9	1	1	Expert
4	Diana	7	1	1	Intermediate
6	Frank	6	1	1	Intermediate
3	Charlie	5	1	1	Novice

Solution: Multi-table join with aggregation demonstrating ER-to-SQL translation.

Q8 Temporal Analysis — Reservation Frequency Ranking**Query:**

```

SELECT s.sid, s.sname,
       COUNT(r.rdate) AS total_reservations,
       MIN(r.rdate) AS first_reservation,
       MAX(r.rdate) AS last_reservation,
       DATEDIFF('DAY', MIN(r.rdate), MAX(r.rdate)) AS activity_span_days,
       RANK() OVER (ORDER BY COUNT(r.rdate) DESC) AS reservation_rank
FROM Sailors s LEFT JOIN Reserves r ON s.sid=r.sid
GROUP BY s.sid, s.sname
ORDER BY reservation_rank;

```

Output:

SID	SNAME	TOTAL_RES	FIRST_RES	LAST_RES	SPAN_DAYS	RANK
2	Bob	2	2026-01-20	2026-03-10	49	1
1	Alice	2	2026-01-15	2026-02-01	17	1
6	Frank	1	2026-02-20	2026-02-20	0	3
4	Diana	1	2026-03-05	2026-03-05	0	3
5	Eve	1	2026-01-30	2026-01-30	0	3

SID	SNAME	TOTAL_RES	FIRST_RES	LAST_RES	SPAN_DAYS	RANK
3	Charlie	1	2026-02-15	2026-02-15	0	3

Solution: Window functions with date arithmetic and ranking; competitive ER-based analytics.

Q9 Constraint Satisfaction — Sailor-Boat Compatibility Analysis

Query:

```
SELECT DISTINCT s.sid, s.sname, b.bid, b.bname, b.boat_type,
COUNT(DISTINCT r.rdate) OVER (PARTITION BY s.sid, b.bid) AS
compat_freq
FROM Sailors s
JOIN Reserves r ON s.sid=r.sid
JOIN Boats b ON r.bid=b.bid
WHERE s.rating >= 6
AND b.boat_type IN ('Sailboat','Yacht')
ORDER BY s.sid, compat_freq DESC;
```

Output:

SID	SNAME	BID	BNAME	BOAT_TYPE	COMPAT_FREQ
1	Alice	101	Nautilus	Sailboat	1
1	Alice	102	Phoenix	Yacht	1
2	Bob	101	Nautilus	Sailboat	1
2	Bob	102	Phoenix	Yacht	1
4	Diana	101	Nautilus	Sailboat	1
6	Frank	104	Emerald	Sailboat	1

Solution: Window-function frequency analysis; constraint-driven sailor-boat pairing.

Q10 UNION — Mutually Exclusive Sailor Categories

Query:

```
SELECT sid, sname, rating, 'High Skill' AS category
FROM Sailors WHERE rating >= 8
UNION ALL
SELECT sid, sname, rating, 'Medium Skill'
FROM Sailors WHERE rating >= 6 AND rating < 8
UNION ALL
SELECT sid, sname, rating, 'Low Skill'
```

FROM Sailors WHERE rating<6
ORDER BY rating DESC;

Output:

SID	SNAME	RATING	CATEGORY
2	Bob	10	High Skill
5	Eve	9	High Skill
1	Alice	8	High Skill
4	Diana	7	Medium Skill
6	Frank	6	Medium Skill
3	Charlie	5	Low Skill

Solution: UNION all categories covering complete sailor set; ER entity classification visualization.

Case Study 2: Nuts & Bolts Supply Chain

Relational Algebra, Joins, Aggregation — Units II-III

Setup — Schema & Data

Create Tables:

```
CREATE TABLE Supplier(sid INT PRIMARY KEY, sname VARCHAR(40),
    sphone VARCHAR(20), location VARCHAR(30));
```

```
CREATE TABLE Product(pid INT PRIMARY KEY, pname VARCHAR(50),
    category VARCHAR(30), standard VARCHAR(20));
```

```
CREATE TABLE Supply(sid INT, pid INT, qty INT, unit_price
    DECIMAL(10,2),
    last_supply_date DATE, PRIMARY KEY(sid,pid),
    FOREIGN KEY(sid) REFERENCES Supplier(sid),
    FOREIGN KEY(pid) REFERENCES Product(pid));
```

Insert Values:

```
INSERT INTO Supplier VALUES
    (501,'FastSupply','9001111','WarehouseA'),
    (502,'QuickParts','9002222','WarehouseB'),
    (503,'BulkNuts','9003333','WarehouseC'),
    (504,'PrecisionBolts','9004444','WarehouseA');
```

```
INSERT INTO Product VALUES
    (1001,'M8 Hex Bolt','Fasteners','ISO4014'),
    (1002,'M10 Carriage Bolt','Fasteners','ISO4015'),
    (1003,'M6 Machine Screw','Fasteners','ISO1207'),
    (1004,'12mm Washer','Fasteners','ISO7091'),
    (1005,'M8 Lock Nut','Fasteners','ISO7040');
```

```
INSERT INTO Supply VALUES
    (501,1001,500,2.50,'2026-02-01'),(501,1002,300,3.00,'2026-02-05'),
    (502,1001,200,2.75,'2026-01-30'),(502,1003,400,1.50,'2026-02-10'),
    (503,1003,600,1.40,'2026-02-08'),(503,1004,800,0.80,'2026-01-25'),
    (504,1005,1000,0.90,'2026-02-12');
```

Supplier Table:

SID	SNAME	SPHONE	LOCATION
501	FastSupply	9001111	WarehouseA
502	QuickParts	9002222	WarehouseB

SID	SNAME	SPHONE	LOCATION
503	BulkNuts	9003333	WarehouseC
504	PrecisionBolts	9004444	WarehouseA

Product Table:

PID	PNAME	CATEGORY	STANDARD
1001	M8 Hex Bolt	Fasteners	ISO4014
1002	M10 Carriage Bolt	Fasteners	ISO4015
1003	M6 Machine Screw	Fasteners	ISO1207
1004	12mm Washer	Fasteners	ISO7091
1005	M8 Lock Nut	Fasteners	ISO7040

Supply Table:

SID	PID	QTY	UNIT_PRICE	LAST_SUPPLY_DATE
501	1001	500	2.5	2026-02-01
501	1002	300	3	2026-02-05
502	1001	200	2.75	2026-01-30
502	1003	400	1.5	2026-02-10
503	1003	600	1.4	2026-02-08
503	1004	800	0.8	2026-01-25
504	1005	1000	0.9	2026-02-12

Q11 Relational Algebra Selection — Suppliers in Specific Location**Query:**

```

SELECT sid, sname
FROM Supplier
WHERE location='WarehouseA'
ORDER BY sname;

```

Output:

SID	SNAME
501	FastSupply
504	PrecisionBolts

Solution: Selection (σ) filters tuples by location predicate.

Q12 Projection — Product Names and Standards

Query:

```
SELECT DISTINCT pname, standard
FROM Product
ORDER BY pname;
```

Output:

PNAME	STANDARD
12mm Washer	ISO7091
M10 Carriage Bolt	ISO4015
M6 Machine Screw	ISO1207
M8 Hex Bolt	ISO4014
M8 Lock Nut	ISO7040

Solution: Projection (π) selects attributes and removes duplicates.

Q13 Cross Product — Supplier-Product Combinations (M-size Only)

Query:

```
SELECT s.sname, p.pname
FROM Supplier s, Product p
WHERE p.pname LIKE 'M%'
ORDER BY s.sname, p.pname;
```

Output:

SNAME	PNAME
BulkNuts	M10 Carriage Bolt
BulkNuts	M6 Machine Screw
BulkNuts	M8 Hex Bolt
BulkNuts	M8 Lock Nut
FastSupply	M10 Carriage Bolt
FastSupply	M6 Machine Screw
FastSupply	M8 Hex Bolt
FastSupply	M8 Lock Nut
PrecisionBolts	M10 Carriage Bolt
PrecisionBolts	M6 Machine Screw

SNAME	PNAME
PrecisionBolts	M8 Hex Bolt
PrecisionBolts	M8 Lock Nut
QuickParts	M10 Carriage Bolt
QuickParts	M6 Machine Screw
QuickParts	M8 Hex Bolt
QuickParts	M8 Lock Nut

Solution: Cartesian product ($\times$) combined with σ for selective cross join.

Q14 Equi-Join — Suppliers with Supplied Products and Quantities

Query:

```
SELECT s.sname, p.pname, sp.qty, sp.unit_price
FROM Supplier s
JOIN Supply sp ON s.sid=sp.sid
JOIN Product p ON sp.pid=p.pid
ORDER BY s.sname, p.pname;
```

Output:

SNAME	PNAME	QTY	UNIT_PRICE
BulkNuts	12mm Washer	800	0.8
BulkNuts	M6 Machine Screw	600	1.4
FastSupply	M10 Carriage Bolt	300	3
FastSupply	M8 Hex Bolt	500	2.5
PrecisionBolts	M8 Lock Nut	1000	0.9
QuickParts	M6 Machine Screw	400	1.5
QuickParts	M8 Hex Bolt	200	2.75

Solution: Equi-join ($\bowtie$) on foreign keys; 3-table join demonstrates relational composition.

Q15 Division — Suppliers Supplying All Product Types

Query:

```
SELECT DISTINCT s.sid, s.sname
FROM Supplier s
WHERE NOT EXISTS (
  SELECT 1 FROM Product p
```

```

WHERE NOT EXISTS (
  SELECT 1 FROM Supply sp
  WHERE sp.sid=s.sid AND sp.pid=p.pid
)
)
ORDER BY s.sid;

```

Output:

SID	SNAME
— (empty set)	No supplier supplies all 5 products

Solution: Division query: universal quantification via NOT EXISTS .

Q16 Union — Combine Suppliers and Products**Query:**

```

SELECT 'Supplier', sid AS id, sname AS name FROM Supplier
UNION
SELECT 'Product', pid, pname FROM Product
ORDER BY 1, 3;

```

Output:

TYPE	ID	NAME
Product	1001	M8 Hex Bolt
Product	1002	M10 Carriage Bolt
Product	1003	M6 Machine Screw
Product	1004	12mm Washer
Product	1005	M8 Lock Nut
Supplier	501	FastSupply
Supplier	502	QuickParts
Supplier	503	BulkNuts
Supplier	504	PrecisionBolts

Solution: UNION combines compatible relations eliminating duplicates.

Q17 Complex Aggregation — Multi-Join with HAVING Clause**Query:**

```

SELECT s.sid, s.sname,

```

```

COUNT(DISTINCT sp.pid) AS product_count,
SUM(sp.qty) AS total_supply,
ROUND(AVG(sp.unit_price),2) AS avg_price,
ROUND(SUM(sp.qty * sp.unit_price),2) AS total_value
FROM Supplier s LEFT JOIN Supply sp ON s.sid=sp.sid
GROUP BY s.sid, s.sname
HAVING COUNT(DISTINCT sp.pid)>=2
ORDER BY total_value DESC;

```

Output:

SID	SNAME	PROD_COUNT	TOTAL_SUPPLY	AVG_PRICE	TOTAL_VALUE
501	Fast Supply	2	800	2.75	2350
503	Bulk Nuts	2	1400	1.1	1480
502	Quick Parts	2	600	2.13	1150

Solution: Group/aggregate with HAVING filter; competitive supplier profiling.

Q18 Conditional Aggregation — High-Volume Products per Supplier

Query:

```

SELECT DISTINCT s.sid, s.sname,
COUNT(DISTINCT sp.pid) FILTER (WHERE sp.qty>300) AS
high_volume_products
FROM Supplier s LEFT JOIN Supply sp ON s.sid=sp.sid
GROUP BY s.sid, s.sname
ORDER BY high_volume_products DESC, s.sname;

```

Output:

SID	SNAME	HIGH_VOL_PRODUCTS
503	BulkNuts	2
501	FastSupply	1
504	PrecisionBolts	1
502	QuickParts	1

Solution: Conditional aggregation with FILTER; competitive supply-chain analysis.

Q19 Self-Join — Supplier Competition on Same Products

Query:

```
SELECT sup1.sname AS supplier_1, sup2.sname AS supplier_2, p.pname,
       s1.unit_price AS price_1, s2.unit_price AS price_2,
       ABS(s1.unit_price - s2.unit_price) AS price_diff
FROM Supply s1
JOIN Supply s2 ON s1.pid=s2.pid AND s1.sid<s2.sid
JOIN Product p ON s1.pid=p.pid
JOIN Supplier sup1 ON s1.sid=sup1.sid
JOIN Supplier sup2 ON s2.sid=sup2.sid
ORDER BY price_diff DESC;
```

Output:

SUPPLIER_1	SUPPLIER_2	PNAME	PRICE_1	PRICE_2	PRICE_DIFF
FastSupply	QuickParts	M8 Hex Bolt	2.5	2.75	0.25
FastSupply	BulkNuts	M6 Machine Screw	1.5	1.40	0.10

Solution: Self-join on supply; competitive pricing analysis.

Q20 CTE + Window Functions — Supplier Ranking by Recency and Volume

Query:

```
WITH supplier_stats AS (
  SELECT s.sid, s.sname,
         COUNT(DISTINCT sp.pid) AS product_types,
         SUM(sp.qty) AS volume,
         MAX(sp.last_supply_date) AS latest_supply,
         RANK() OVER (ORDER BY MAX(sp.last_supply_date) DESC) AS
recency_rank,
         PERCENT_RANK() OVER (ORDER BY SUM(sp.qty) DESC) AS vol_pct
  FROM Supplier s LEFT JOIN Supply sp ON s.sid=sp.sid
  GROUP BY s.sid, s.sname
)
SELECT sid, sname, product_types, volume, latest_supply,
       recency_rank, ROUND(vol_pct*100,1) AS vol_pct_score
FROM supplier_stats
ORDER BY recency_rank, volume DESC;
```

Output:

SID	SNAME	PROD_TYPES	VOLUME	LATEST_SUPPLY	REC_RANK	VOL_PCT
504	Precision Bolts	1	1000	2026-02-12	1	33.3
502	Quick Parts	2	600	2026-02-10	2	100.0
503	Bulk Nuts	2	1400	2026-02-08	3	0.0
501	Fast Supply	2	800	2026-02-05	4	66.7

Solution: CTE + window functions for multi-dimensional supplier analytics; competitive exam standard.

Case Study 3: Vegetables Market Analytics

Advanced SQL, Subqueries, Window Functions — Units II-III

Setup — Schema & Data

Create Tables:

```
CREATE TABLE Farmer(fid INT PRIMARY KEY, fname VARCHAR(40),  
    location VARCHAR(30), registration_date DATE);
```

```
CREATE TABLE VegetableType(vid INT PRIMARY KEY, vname  
    VARCHAR(40),  
    crop_family VARCHAR(30), season VARCHAR(20));
```

```
CREATE TABLE Production(fid INT, vid INT, harvest_date DATE,  
    qty_kg INT, market_price DECIMAL(7,2),  
    PRIMARY KEY(fid,vid,harvest_date),  
    FOREIGN KEY(fid) REFERENCES Farmer(fid),  
    FOREIGN KEY(vid) REFERENCES VegetableType(vid));
```

Tables:

FID	FNAME	LOCATION	REGISTRATION_DATE
1	Ramesh	Valley	2023-01-15
2	Priya	Hills	2023-06-20
3	Vikram	Valley	2022-12-10
4	Anita	Coast	2024-02-28

VID	VNAME	CROP_FAMILY	SEASON
101	Tomato	Solanaceae	Summer
102	Potato	Solanaceae	Winter
103	Carrot	Apiaceae	Winter
104	Broccoli	Brassicaceae	Winter
105	Lettuce	Asteraceae	Spring

FID	VID	HARVEST_DATE	QTY_KG	MARKET_PRICE
1	101	2026-03-15	2000	40
1	102	2026-01-10	3500	20
2	103	2026-01-20	1500	35
2	104	2026-02-05	800	50
3	101	2026-03-20	1800	42
3	105	2026-04-10	1200	60
4	102	2026-02-15	2500	22
4	105	2026-04-15	900	65

Q21 Simple Subquery — Vegetables with Above-Average Production

Query:

```
SELECT v.vname, p.qty_kg
FROM VegetableType v JOIN Production p ON v.vid=p.vid
WHERE p.qty_kg > (SELECT AVG(qty_kg) FROM Production)
ORDER BY p.qty_kg DESC;
```

Output:

VNAME	QTY_KG
Potato	3500
Potato	2500
Tomato	2000
Tomato	1800

Solution: Subquery computes global average; outer query filters above-average harvests.

Q22 Correlated Subquery — Farmers with Above-Location-Average Price

Query:

```
SELECT f.fname, f.location, v.vname, p.market_price
FROM Farmer f
JOIN Production p ON f.fid=p.fid
JOIN VegetableType v ON p.vid=v.vid
WHERE p.market_price > (
  SELECT AVG(market_price) FROM Production
```

```

WHERE (SELECT location FROM Farmer WHERE
fid=Production.fid)=f.location
)
ORDER BY f.location, p.market_price DESC;

```

Output:

FNAME	LOCATION	VNAME	MARKET_PRICE
Anita	Coast	Lettuce	65.00
Priya	Hills	Broccoli	50.00
Vikram	Valley	Lettuce	60.00
Vikram	Valley	Tomato	42.00

Solution: Correlated subquery references outer farmer location for per-location filtering.

Q23 IN Operator — Farmers Producing Winter Crops**Query:**

```

SELECT DISTINCT f.fid, f.fname
FROM Farmer f
WHERE f.fid IN (
  SELECT p.fid FROM Production p
  WHERE p.vid IN (
    SELECT vid FROM VegetableType WHERE season='Winter'
  )
)
ORDER BY f.fname;

```

Output:

FID	FNAME
4	Anita
2	Priya
1	Ramesh

Solution: Nested IN subqueries for multi-level filtering on crop families.

24 EXISTS — Farmers Who Produced Tomato AND Potato**Query:**

```

SELECT f.fid, f.fname
FROM Farmer f

```

```
WHERE EXISTS (SELECT 1 FROM Production p WHERE p.fid=f.fid AND
p.vid=101)
AND EXISTS (SELECT 1 FROM Production p WHERE p.fid=f.fid AND
p.vid=102)
ORDER BY f.fname;
```

Output:

FID	FNAME
1	Ramesh

Solution: Multiple EXISTS subqueries enforce AND logic on farmer production.

Q25 Window RANK — Farmers by Total Revenue Within Location**Query:**

```
SELECT f.fid, f.fname, f.location,
ROUND(SUM(p.qty_kg * p.market_price),2) AS total_revenue,
RANK() OVER (PARTITION BY f.location
ORDER BY SUM(p.qty_kg*p.market_price) DESC) AS
location_rank
FROM Farmer f JOIN Production p ON f.fid=p.fid
GROUP BY f.fid, f.fname, f.location
ORDER BY f.location, location_rank;
```

Output:

FID	FNAME	LOCATION	TOTAL_REVENUE	LOCATION_RANK
4	Anita	Coast	113500.00	1
2	Priya	Hills	92500.00	1
3	Vikram	Valley	147600.00	1
1	Ramesh	Valley	150000.00	2

Solution: Window RANK() partitions farmers by location for competitive revenue analysis.

Q26 LAG/LEAD — Price Trends Across Harvests**Query:**

```
SELECT f.fname, v.vname, p.harvest_date, p.market_price,
LAG(p.market_price) OVER (PARTITION BY f.fid, p.vid ORDER BY
p.harvest_date) AS prev_price,
LEAD(p.market_price) OVER (PARTITION BY f.fid, p.vid ORDER BY
p.harvest_date) AS next_price,
```

```

ROUND(100.0*(p.market_price - LAG(p.market_price) OVER (
    PARTITION BY f.fid, p.vid ORDER BY p.harvest_date))
/ LAG(p.market_price) OVER (
    PARTITION BY f.fid, p.vid ORDER BY p.harvest_date), 2) AS
price_chg_pct
FROM Farmer f JOIN Production p ON f.fid=p.fid
JOIN VegetableType v ON p.vid=v.vid
ORDER BY f.fname, v.vid, p.harvest_date;

```

Output:

FNAME	VNAME	HARVEST_ DATE	MARKET_ PRICE	PREV_ PRICE	NEXT_ PRICE	PRICE_ CHG%
Anita	Lettuce	2026-04-15	65	NULL	NULL	NULL
Anita	Potato	2026-02-15	22	NULL	NULL	NULL
Priya	Broccoli	2026-02-05	50	NULL	NULL	NULL
Priya	Carrot	2026-01-20	35	NULL	NULL	NULL
Ramesh	Potato	2026-01-10	20	NULL	NULL	NULL
Ramesh	Tomato	2026-03-15	40	NULL	NULL	NULL
Vikram	Lettuce	2026-04-10	60	NULL	NULL	NULL
Vikram	Tomato	2026-03-20	42	NULL	NULL	NULL

Solution: LAG/LEAD window functions detect price variations over sequential harvests.

Q27 CTE with Window Ranking — Seasonal Crop Production Analysis

Query:

```

WITH seasonal_analysis AS (
    SELECT v.season, f.location,
           COUNT(DISTINCT f.fid) AS farmer_count,
           SUM(p.qty_kg) AS total_qty,
           AVG(p.market_price) AS avg_price
    FROM VegetableType v
    LEFT JOIN Production p ON v.vid=p.vid
    LEFT JOIN Farmer f ON p.fid=f.fid
    GROUP BY v.season, f.location
),
ranked_seasons AS (
    SELECT season, location, farmer_count, total_qty,

```

```

ROUND(avg_price,2) AS avg_price,
RANK() OVER (PARTITION BY season ORDER BY total_qty DESC) AS
volume_rank
FROM seasonal_analysis WHERE total_qty IS NOT NULL
)
SELECT season, location, farmer_count, total_qty, avg_price, volume_rank
FROM ranked_seasons
ORDER BY season, volume_rank;

```

Output:

SEASON	LOCATION	FARMER_COUNT	TOTAL_QTY	AVG_PRICE	VOL_RANK
Spring	Valley	1	1200	60.00	1
Spring	Coast	1	900	65.00	2
Summer	Valley	2	3800	41.00	1
Winter	Valley	1	3500	20.00	1
Winter	Coast	1	2500	22.00	2
Winter	Hills	1	2300	42.50	3

Solution: CTE + window ranking for seasonal production portfolio analysis.

Q28 Multi-Level Aggregation — Region Analysis with HAVING Subquery

Query:

```

SELECT f.location,
COUNT(DISTINCT f.fid) AS active_farmers,
COUNT(DISTINCT v.vid) AS vegetable_variety,
ROUND(AVG(p.market_price),2) AS region_avg_price,
ROUND(SUM(p.qty_kg * p.market_price),0) AS region_total_value
FROM Farmer f
JOIN Production p ON f.fid=p.fid
JOIN VegetableType v ON p.vid=v.vid
GROUP BY f.location
HAVING AVG(p.market_price) > (SELECT AVG(market_price) FROM
Production)
AND SUM(p.qty_kg) > (SELECT AVG(qty_kg)*5 FROM Production)
ORDER BY region_total_value DESC;

```

Output:

LOCATION	ACTIVE_FARMERS	VEG_VARIETY	AVG_PRICE	TOTAL_VALUE
—	(empty set — no location passes both HAVING conditions)			

Solution: HAVING with aggregate subqueries; multi-dimensional region-level analysis.

Q29 CASE WHEN Pivot — Farmer-Vegetable Production Matrix

Query:

```

SELECT f.fname, f.location,
       SUM(CASE WHEN v.vname='Tomato' THEN p.qty_kg ELSE 0 END) AS
tomato_qty,
       SUM(CASE WHEN v.vname='Potato' THEN p.qty_kg ELSE 0 END) AS
potato_qty,
       SUM(CASE WHEN v.vname='Carrot' THEN p.qty_kg ELSE 0 END) AS
carrot_qty,
       SUM(CASE WHEN v.vname='Broccoli' THEN p.qty_kg ELSE 0 END) AS
broccoli_qty,
       SUM(CASE WHEN v.vname='Lettuce' THEN p.qty_kg ELSE 0 END) AS
lettuce_qty,
       COUNT(DISTINCT v.vid) AS crop_variety
FROM Farmer f
LEFT JOIN Production p ON f.fid=p.fid
LEFT JOIN VegetableType v ON p.vid=v.vid
GROUP BY f.fid, f.fname, f.location
ORDER BY f.location, f.fname;

```

Output:

FNAME	LOCATION	TOMATO	POTATO	CARROT	BROCCOLI	LETTUCE	VARIETY
Anita	Coast	0	2500	0	0	900	2
Priya	Hills	0	0	1500	800	0	2
Ramesh	Valley	2000	3500	0	0	0	2
Vikram	Valley	1800	0	0	0	1200	2

Solution: Conditional aggregation (CASE WHEN) for production matrix; competitive exam analytical technique.

Q30 Statistical Aggregation — Production Volatility Analysis

Query:

```
SELECT f.fname, v.vname,
       COUNT(DISTINCT DATE_TRUNC('month',p.harvest_date)) AS
active_months,
       MIN(p.harvest_date) AS first_harvest,
       MAX(p.harvest_date) AS last_harvest,
       ROUND(STDDEV_POP(p.qty_kg),2) AS production_variance,
       ROUND(100.0*MAX(p.qty_kg)/NULLIF(MIN(p.qty_kg),0)-100,2) AS
max_growth_pct
FROM Farmer f JOIN Production p ON f.fid=p.fid
JOIN VegetableType v ON p.vid=v.vid
GROUP BY f.fid, f.fname, v.vid, v.vname
HAVING COUNT(DISTINCT DATE_TRUNC('month',p.harvest_date))>1
       OR STDDEV_POP(p.qty_kg)>0
ORDER BY max_growth_pct DESC;
```

Output:

FNAME	VNAME	ACTIVE_MONTHS	FIRST_HARVEST	LAST_HARVEST	VARIANCE	MAX_GROWTH%
Anita	Lettuce	1	2026-04-15	2026-04-15	0	0
Anita	Potato	1	2026-02-15	2026-02-15	0	0
Priya	Broccoli	1	2026-02-05	2026-02-05	0	0
Priya	Carrot	1	2026-01-20	2026-01-20	0	0
Ramesh	Potato	1	2026-01-10	2026-01-10	0	0
Ramesh	Tomato	1	2026-03-15	2026-03-15	0	0
Vikram	Lettuce	1	2026-04-10	2026-04-10	0	0
Vikram	Tomato	1	2026-03-20	2026-03-20	0	0

Solution: Statistical aggregates (STDDEV_POP) for production volatility; competitive forecasting analysis.

Case Study 4: Tuple Relational Calculus (TRC) & Domain Relational Calculus (DRC)

Tuple Relational Calculus (TRC)

TRC is a non-procedural query language where queries describe **WHAT** to retrieve, not HOW. A TRC expression has the form:

$$\{t \mid P(t)\}$$

Where t = tuple variable

$P(t)$ = predicate (condition) that t must satisfy

Quantifiers used in TRC:

Symbol	Name	Meaning
$\forall$	Universal Quantifier (FOR ALL)	The predicate holds for every tuple
$\exists$	Existential Quantifier (THERE EXISTS)	At least one tuple satisfies the predicate
$\wedge$	AND	Both conditions must be true
$\vee$	OR	At least one condition must be true
$\neg$	NOT	Negation of the predicate
$\Rightarrow$	IMPLIES	If condition A then condition B

Domain Relational Calculus (DRC)

DRC uses **domain variables** (one per attribute) instead of tuple variables. A DRC expression has the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

where $x_1..x_n$ = domain variables (one per column)

P = predicate referencing those variables

Key difference: TRC binds a full tuple (t.attr); DRC binds individual attribute domains (x, y, z...).

Shared Schema: Sailors · Boats · Reserves

Sailors

SID	SNAME	RATING	AGE
1	Alice	8	45.0
2	Bob	10	63.5
3	Charlie	5	35.0
4	Diana	7	42.0
5	Eve	9	51.0
6	Frank	6	38.5

Boats

BID	BNAME	COLOR	BOAT_TYPE
101	Nautilus	Red	Sailboat
102	Phoenix	Green	Yacht
103	Titan	Blue	Speedboat
104	Emerald	Green	Sailboat

Reserves

SID	BID	RDATE
1	101	2026-01-15
1	102	2026-02-01
2	101	2026-01-20
2	102	2026-03-10
3	104	2026-02-15
4	101	2026-03-05
5	103	2026-01-30
6	104	2026-02-20

Tuple Relational Calculus (TRC)**Basic Selection — Sailors with Rating > 7****TRC Expression:**

$$\{ t \mid \text{Sailors}(t) \wedge t.\text{rating} > 7 \}$$

Read: Return tuple t such that t is in Sailors AND $t.\text{rating} > 7$

SQL Equivalent:

```
SELECT * FROM Sailors
```

```
WHERE rating > 7;
```

Result:

SID	SNAME	RATING	AGE
2	Bob	10	63.5

SID	SNAME	RATING	AGE
5	Eve	9	51.0
1	Alice	8	45.0

Solution: The predicate Sailors(t) restricts t to tuples in the Sailors relation; $t.rating > 7$ is the selection condition. Equivalent to $\sigma(rating > 7)(Sailors)$ in relational algebra.

Note: t is a tuple variable that ranges over all tuples in Sailors.

Projection — Names and Ratings of All Sailors

TRC Expression:

$\{t.sname, t.rating \mid Sailors(t)\}$

Read: Return sname and rating for every tuple t in Sailors

SQL Equivalent:

SELECT sname, rating FROM Sailors;

Result:

SNAME	RATING
Alice	8
Bob	10
Charlie	5
Diana	7
Eve	9
Frank	6

Solution: Projection in TRC: list only the desired attributes (t.sname, t.rating). All tuples t in Sailors are included — no filter predicate.

Note: TRC projection lists specific attributes before the pipe '|'.
Duplicates are implicitly removed (like DISTINCT).

Selection with Compound Predicate (AND) — Expert Sailors

TRC Expression:

{ t.sname, t.rating, t.age | Sailors(t) $\wedge$ t.rating $\geq$ 8 $\wedge$ t.age $<$ 60 }

Read: Sailors with rating $\geq$ 8 AND age $<$ 60

SQL Equivalent:

SELECT sname, rating, age FROM Sailors

WHERE rating $\geq$ 8 AND age $<$ 60;

Result:

SNAME	RATING	AGE
Alice	8	45.0
Eve	9	51.0

Solution: Multiple conditions joined by $\wedge$ (AND) in TRC. Both predicates must hold simultaneously. Bob (age 63.5) is excluded by the age filter.

Note: $\wedge$ in TRC maps directly to AND in SQL WHERE clause.

Selection with OR — Sailors with Very High or Very Low Rating

TRC Expression:

{ t.sname, t.rating | Sailors(t) $\wedge$ (t.rating = 10 $\vee$ t.rating $<$ 6) }

Read: Sailors whose rating equals 10 OR is less than 6

SQL Equivalent:

SELECT sname, rating FROM Sailors

WHERE rating = 10 OR rating $<$ 6;

Result:

SNAME	RATING
Bob	10
Charlie	5

Solution: Disjunction ($\vee$ / OR) in TRC: either condition being true is sufficient. Parentheses group the OR clause within the larger predicate.

Note: $\vee$ in TRC maps to OR in SQL. Use parentheses carefully when mixing $\wedge$ and $\vee$ to avoid precedence issues.

Existential Quantifier — Sailors Who Have Reserved at Least One Boat**TRC Expression:**

$$\{ t \mid \text{Sailors}(t) \wedge \exists r (\text{Reserves}(r) \wedge r.\text{sid} = t.\text{sid}) \}$$

Read: Return tuple t from Sailors such that

there EXISTS a tuple r in Reserves where $r.\text{sid} = t.\text{sid}$

SQL Equivalent:

```
SELECT * FROM Sailors s
WHERE EXISTS (
  SELECT 1 FROM Reserves r WHERE r.sid = s.sid
);
```

Result:

SID	SNAME	RATING	AGE
1	Alice	8	45.0
2	Bob	10	63.5
3	Charlie	5	35.0

SID	SNAME	RATING	AGE
4	Diana	7	42.0
5	Eve	9	51.0
6	Frank	6	38.5

Solution: The existential quantifier $\exists r$ introduces a new tuple variable r ranging over Reserves. The join condition $r.sid = t.sid$ links the two relations. Here all sailors have at least one reservation.

Note: $\exists$ in TRC maps to EXISTS in SQL. The bound variable r is local to the quantifier scope.

Existential + Join — Names of Sailors Who Reserved a Red Boat

TRC Expression:

$$\{ t.sname \mid \text{Sailors}(t) \wedge \\ \exists r (\text{Reserves}(r) \wedge r.sid = t.sid \wedge \\ \exists b (\text{Boats}(b) \wedge b.bid = r.bid \wedge b.color = 'Red')) \}$$

Read: Sailor names where $\exists$ reservation linking to $\exists$ red boat

SQL Equivalent:

```
SELECT DISTINCT s.sname FROM Sailors s
JOIN Reserves r ON s.sid = r.sid
JOIN Boats b ON r.bid = b.bid
WHERE b.color = 'Red';
```

Result:

SNAME
Alice
Bob

SNAME

Diana

Solution: Nested existential quantifiers simulate a three-table join in TRC. The outermost $\exists r$ finds a reservation; the inner $\exists b$ checks the boat's color. Each $\exists$ introduces a new bound variable.

Note: Three-relation TRC queries always use nested $\exists$ quantifiers — one per intermediate relation.

[Unit II]

Negation (NOT) — Sailors Who Never Reserved Any Boat**TRC Expression:**

$$\{ t \mid \text{Sailors}(t) \wedge \neg \exists r (\text{Reserves}(r) \wedge r.\text{sid} = t.\text{sid}) \}$$

Read: Sailors for whom there does NOT EXIST any reservation tuple

SQL Equivalent:

```
SELECT * FROM Sailors s
WHERE NOT EXISTS (
  SELECT 1 FROM Reserves r WHERE r.sid = s.sid
);
```

Result:

SID	SNAME	RATING	AGE
—	(empty set)	—	—

Solution: $\neg \exists$ (negation of existential) implements NOT EXIST in TRC. Since all 6 sailors have reservations, the result is empty. $\neg \exists r P(r)$ is logically equivalent to $\forall r \neg P(r)$.

Note: $\neg \exists$ and NOT EXISTS are logically equivalent. This is the standard TRC pattern for anti-join queries.

Universal Quantifier — Sailors Who Reserved ALL Boats (Relational Division)

TRC Expression:

$$\{ t \mid \text{Sailors}(t) \wedge \\ \forall b (\text{Boats}(b) \Rightarrow \\ \exists r (\text{Reserves}(r) \wedge r.\text{sid} = t.\text{sid} \wedge r.\text{bid} = b.\text{bid})) \}$$

Read: Sailors t such that for every boat b ,
there exists a reservation by t for b

SQL Equivalent:

```
SELECT s.sid, s.sname FROM Sailors s
WHERE NOT EXISTS (
  SELECT b.bid FROM Boats b
  WHERE NOT EXISTS (
    SELECT 1 FROM Reserves r
    WHERE r.sid = s.sid AND r.bid = b.bid
  )
);
```

Result:

SID	SNAME
—	(empty set — no sailor reserved all 4 boats)

Solution: Universal quantification ($\forall$) implements relational division. $\forall b P(b)$ means the predicate must hold for every tuple in Boats. In SQL this is expressed as double NOT EXISTS (no boat exists for which there is no reservation).

Note: $\forall b (A \Rightarrow B)$ is equivalent to $\neg \exists b (A \wedge \neg B)$. This duality lets you convert universal TRC to SQL NOT EXISTS.

Multi-Attribute Projection with Condition — Boat Names Reserved in January 2026

TRC Expression:

$$\{ b.bname, b.color \mid Boats(b) \wedge \\ \exists r (Reserves(r) \wedge r.bid = b.bid \\ \wedge r.rdate \geq '2026-01-01' \\ \wedge r.rdate \leq '2026-01-31') \}$$

Read: Boat name and color where $\exists$ a January 2026 reservation

SQL Equivalent:

```
SELECT DISTINCT b.bname, b.color FROM Boats b
JOIN Reserves r ON b.bid = r.bid
WHERE r.rdate BETWEEN '2026-01-01' AND '2026-01-31';
```

Result:

BNAME	COLOR
Nautilus	Red
Titan	Blue

Solution: Date range conditions are ordinary predicates in TRC. The $\exists r$ quantifier joins Boats and Reserves; the date predicates on r filter the reservation period.

Note: String and date comparisons in TRC are written exactly as in SQL predicates.

Implication (Q) — Sailors Where Every Reservation is for a Green Boat

TRC Expression:

$$\{ t \mid Sailors(t) \wedge \\ \forall r (Reserves(r) \wedge r.sid = t.sid \Rightarrow \\ \exists b (Boats(b) \wedge b.bid = r.bid \wedge b.color = 'Green')) \}$$

Read: Sailors t such that ALL their reservations are for green boats

SQL Equivalent:

```

SELECT s.sid, s.sname FROM Sailors s
WHERE NOT EXISTS (
  SELECT 1 FROM Reserves r
  JOIN Boats b ON r.bid = b.bid
  WHERE r.sid = s.sid AND b.color <> 'Green'
);

```

Result:

SID	SNAME
3	Charlie
6	Frank

Solution: Implication $A \Rightarrow B$ means: if a sailor has a reservation (A), it must be for a green boat (B). In SQL: no reservation exists for a non-green boat. Charlie (boat 104=Emerald/Green) and Frank (boat 104) qualify.

Note: $\forall r (A \Rightarrow B)$ is equivalent to $\neg \exists r (A \wedge \neg B)$. Implication is the standard tool for 'all reservations satisfy property P' queries.

Conjunction of Existentials — Sailors Who Reserved Both a Sailboat and a Yacht**TRC Expression:**

```

{ t.sname | Sailors(t) ∧
  ∃r1 ∃b1 (Reserves(r1) ∧ Boats(b1) ∧ r1.sid=t.sid ∧ r1.bid=b1.bid
    ∧ b1.boat_type='Sailboat') ∧
  ∃r2 ∃b2 (Reserves(r2) ∧ Boats(b2) ∧ r2.sid=t.sid ∧ r2.bid=b2.bid
    ∧ b2.boat_type='Yacht')}

```

Read: Sailors who have $\exists$ reservation for a Sailboat AND $\exists$ reservation for a Yacht

SQL Equivalent:

```

SELECT s.sname FROM Sailors s

```

```

WHERE EXISTS (SELECT 1 FROM Reserves r JOIN Boats b ON r.bid=b.bid
              WHERE r.sid=s.sid AND b.boat_type='Sailboat')
AND EXISTS (SELECT 1 FROM Reserves r JOIN Boats b ON r.bid=b.bid
              WHERE r.sid=s.sid AND b.boat_type='Yacht');

```

Result:

SNAME
Alice
Bob

Solution: Two independent existential quantifiers with different bound variables (r1,b1) and (r2,b2) express that BOTH conditions must hold. Alice and Bob each reserved Nautilus (Sailboat) and Phoenix (Yacht).

Note: Use distinct variable names (r1,r2,b1,b2) for each independent existential; reusing names would incorrectly link the two conditions.

Negation of Conjunction — Sailors Who Did NOT Reserve Boat 101**TRC Expression:**

$$\{ t \mid \text{Sailors}(t) \wedge \neg \exists r (\text{Reserves}(r) \wedge r.\text{sid}=t.\text{sid} \wedge r.\text{bid}=101) \}$$

Read: Sailors for whom there is NO reservation for bid=101

SQL Equivalent:

```

SELECT * FROM Sailors s
WHERE NOT EXISTS (
  SELECT 1 FROM Reserves r
  WHERE r.sid = s.sid AND r.bid = 101
);

```

Result:

SID	SNAME	RATING	AGE
3	Charlie	5	35.0
5	Eve	9	51.0
6	Frank	6	38.5

Solution: $\neg\exists r$ with an added equality condition on $\text{bid}=101$ implements NOT IN / NOT EXISTS for a specific boat. Charlie (104), Eve (103), and Frank (104) never reserved boat 101.

Note: This is the TRC anti-join pattern: $\neg\exists$ combined with a join condition and a filter attribute.

Nested Quantifiers — Boats Reserved by Sailors with Rating = 10**TRC Expression:**

$$\{ b.\text{bname}, b.\text{boat_type} \mid \text{Boats}(b) \wedge \\ \exists r (\text{Reserves}(r) \wedge r.\text{bid}=b.\text{bid} \wedge \\ \exists s (\text{Sailors}(s) \wedge s.\text{sid}=r.\text{sid} \wedge s.\text{rating}=10)) \}$$

Read: Boats reserved by $\exists$ sailor with rating 10

SQL Equivalent:

```
SELECT DISTINCT b.bname, b.boat_type FROM Boats b
JOIN Reserves r ON b.bid=r.bid
JOIN Sailors s ON r.sid=s.sid
WHERE s.rating = 10;
```

Result:

BNAME	BOAT_TYPE
Nautilus	Sailboat

BNAME	BOAT_TYPE
Phoenix	Yacht

Solution: Triple nested quantifiers: $\exists b$ scoped by the outer Boats(b), then $\exists r$ for Reserves, then $\exists s$ for Sailors. Only Bob (rating=10) and his reserved boats Nautilus and Phoenix are returned.

Note: The outermost variable b is the free variable appearing in the result set. All other variables (r, s) are bound by $\exists$.

Competitive — Sailors Who Reserved Only Green Boats (No Non-Green Reservations)

TRC Expression:

$$\{ t \mid \text{Sailors}(t) \wedge \exists r (\text{Reserves}(r) \wedge r.\text{sid}=t.\text{sid}) \wedge \\ \neg \exists r2 \exists b (\text{Reserves}(r2) \wedge \text{Boats}(b) \wedge \\ r2.\text{sid}=t.\text{sid} \wedge r2.\text{bid}=b.\text{bid} \wedge b.\text{color} \neq \text{'Green'}) \}$$

Read: Sailors who HAVE at least one reservation AND have NO reservation for a non-green boat

SQL Equivalent:

```
SELECT s.sid, s.sname FROM Sailors s
WHERE EXISTS (SELECT 1 FROM Reserves r WHERE r.sid=s.sid)
AND NOT EXISTS (
    SELECT 1 FROM Reserves r JOIN Boats b ON r.bid=b.bid
    WHERE r.sid=s.sid AND b.color <> 'Green'
);
```

Result:

SID	SNAME
3	Charlie
6	Frank

Solution: Two-part predicate: (1) $\exists$ at least one reservation, (2) $\neg\exists$ any reservation for a non-green boat. The conjunction ensures the sailor is active AND all their boats are green. Charlie and Frank only reserved boat 104 (Emerald, Green).

Note: The first $\exists$ avoids vacuous truth — without it, sailors with NO reservations would also satisfy $\neg\exists$ non-green reservations.

Competitive — TRC for Set Intersection (Boats Reserved in Both Jan and Feb 2026)

TRC Expression:

$$\{ b \mid \text{Boats}(b) \wedge \\ \exists r1 (\text{Reserves}(r1) \wedge r1.\text{bid}=b.\text{bid} \\ \wedge r1.\text{rdate} \geq '2026-01-01' \wedge r1.\text{rdate} \leq '2026-01-31') \wedge \\ \exists r2 (\text{Reserves}(r2) \wedge r2.\text{bid}=b.\text{bid} \\ \wedge r2.\text{rdate} \geq '2026-02-01' \wedge r2.\text{rdate} \leq '2026-02-28') \}$$

Read: Boats with $\exists$ reservation in January AND $\exists$ reservation in February

SQL Equivalent:

```
SELECT DISTINCT b.* FROM Boats b
WHERE b.bid IN (SELECT bid FROM Reserves WHERE rdate BETWEEN
'2026-01-01' AND '2026-01-31')
AND b.bid IN (SELECT bid FROM Reserves WHERE rdate BETWEEN '2026-
02-01' AND '2026-02-28');
```

Result:

BID	BNAME	COLOR	BOAT_TYPE
101	Nautilus	Red	Sailboat

Solution: Set intersection via TRC: two independent existentials r1 and r2 with different date ranges. Both must hold for the same boat b. Only Nautilus (101) was reserved in both January and February.

Note: TRC naturally expresses intersection as $\exists r1 (...Jan...) \wedge \exists r2 (...Feb...)$ — no INTERSECT keyword needed.

Competitive — Sailors Whose Rating is Above All Others in a Group**TRC Expression:**

$$\{ t \mid \text{Sailors}(t) \wedge \neg \exists s (\text{Sailors}(s) \wedge s.\text{rating} > t.\text{rating}) \}$$

Read: Sailors t for whom NO other sailor s has a strictly higher rating

(i.e., t has the maximum rating)

SQL Equivalent:

```
SELECT * FROM Sailors s
```

```
WHERE rating = (SELECT MAX(rating) FROM Sailors);
```

Result:

SID	SNAME	RATING	AGE
2	Bob	10	63.5

Solution: MAX in TRC: $\neg \exists s (\text{Sailors}(s) \wedge s.\text{rating} > t.\text{rating})$ means no sailor exists with a higher rating than t. This is the standard TRC pattern for finding the maximum value.

Note: TRC has no aggregate functions (SUM, MAX, COUNT). Aggregation is expressed through quantifier patterns like $\neg \exists$ for MAX and $\forall$ for ALL.

Competitive — Cross-Table: Sailors Younger Than All Sailors Who Reserved Boat 101**TRC Expression:**
$$\{ t \mid \text{Sailors}(t) \wedge \forall s (\text{Sailors}(s) \wedge \exists r (\text{Reserves}(r) \wedge r.\text{sid}=s.\text{sid} \wedge r.\text{bid}=101) \Rightarrow t.\text{age} < s.\text{age}) \}$$

Read: Sailors t such that for every sailor s who reserved boat 101,
 t .age is less than s .age

SQL Equivalent:

```
SELECT * FROM Sailors
WHERE age < ALL (
  SELECT s.age FROM Sailors s
  JOIN Reserves r ON s.sid=r.sid
  WHERE r.bid=101
);
```

Result:

SID	SNAME	RATING	AGE
3	Charlie	5	35.0

Solution: $\forall s (P(s) \Rightarrow Q(t,s))$ expresses 'for all sailors satisfying P , t satisfies Q relative to s '. Reservers of boat 101 are Alice(45), Bob(63.5), Diana(42). Charlie(35) is younger than all three.

Note: The implication $A \Rightarrow B$ ensures we only constrain t against sailors who actually reserved boat 101, not all sailors.

Competitive — Safety: Safe vs Unsafe TRC Expressions**TRC Expression:**

-- UNSAFE (infinite result): $\{ t \mid \neg \text{Sailors}(t) \}$

Read: All tuples NOT in Sailors — infinite domain!

-- SAFE version using domain restriction:

$\{ t \mid \text{Sailors}(t) \wedge \neg \exists r (\text{Reserves}(r) \wedge r.\text{sid} = t.\text{sid}) \}$

Rule: Every free variable must be domain-restricted to a finite relation.

SQL Equivalent:

SELECT * FROM Sailors s

WHERE NOT EXISTS (SELECT 1 FROM Reserves r WHERE r.sid=s.sid);

Result:

CONCEPT	DESCRIPTION
Unsafe TRC	$\{ t \mid \neg \text{Sailors}(t) \}$ — result is infinite (all non-Sailor tuples in domain)
Safe TRC	Every variable appears in a positive relational atom in the predicate
Domain Restriction	t must appear in $\text{Sailors}(t)$ or some other base relation in the query
SQL Safety	SQL is always safe by design — FROM clause restricts domains

Solution: A TRC expression is safe if every value in the result is drawn from the active domain (values appearing in the database). Unsafe expressions produce infinite results — they are syntactically valid but semantically unusable.

Note: Exam tip: A TRC expression is safe if and only if every free variable t appears in at least one positive relational atom like $Sailors(t)$, $Boats(t)$, or $Reserves(t)$.

Competitive — TRC for Set Difference (Sailors Who Never Reserved a Speedboat)

TRC Expression:

$$\{ t \mid Sailors(t) \wedge \\ \neg \exists r \exists b (Reserves(r) \wedge Boats(b) \wedge \\ r.sid = t.sid \wedge r.bid = b.bid \wedge b.boat_type = 'Speedboat') \}$$

Read: Sailors for whom no reservation exists for any Speedboat

SQL Equivalent:

```
SELECT * FROM Sailors
WHERE sid NOT IN (
  SELECT DISTINCT r.sid FROM Reserves r
  JOIN Boats b ON r.bid=b.bid
  WHERE b.boat_type='Speedboat'
);
```

Result:

SID	SNAME	RATING	AGE
1	Alice	8	45.0
2	Bob	10	63.5
3	Charlie	5	35.0
4	Diana	7	42.0
6	Frank	6	38.5

Solution: Set difference via TRC: $\neg\exists r\exists b$ with `boat_type='Speedboat'`. Only Eve (`sid=5`) reserved Titan (`Speedboat`, `bid=103`), so she is excluded. All others qualify.

Note: Set difference in TRC uses $\neg\exists$. The equivalent relational algebra is `Sailors -  $\pi_{sid}(\text{Reserves} \bowtie \sigma(\text{boat\_type}='Speedboat')(\text{Boats}))$` .

Competitive — TRC Equivalent of Aggregation (Sailors with Above-Average Rating)

TRC Expression:

`{ t | Sailors(t)  $\wedge$`
 `$\neg\exists s_1 \exists s_2 \dots$  (complex count expression — TRC cannot express AVG directly)`
 -- In practice, approximate via comparison with a known threshold:
`{ t | Sailors(t)  $\wedge$  t.rating > 7 }` -- Approximation for 'above average'
 -- Exact average (7.25) requires arithmetic outside pure TRC:
 -- This illustrates a LIMITATION of TRC.

SQL Equivalent:

```
SELECT * FROM Sailors
WHERE rating > (SELECT AVG (rating) FROM Sailors);
-- AVG = (8+10+5+7+9+6)/6 = 7.5
```

Result:

SID	SNAME	RATING	LIMITATION
1	Alice	8	TRC can compare <code>t.rating > constant</code> but cannot compute AVG
2	Bob	10	Aggregate functions (SUM, AVG, COUNT) are beyond pure TRC/DRC
5	Eve	9	SQL extends relational calculus with GROUP BY / aggregation

Solution: Pure TRC (and DRC) cannot express aggregate functions. This is a fundamental limitation: TRC is equivalent in expressive power to First-Order Logic (FOL), which cannot count. SQL's GROUP BY and aggregate functions are extensions beyond FOL.

Note: Exam tip: TRC/DRC expressiveness = Relational Algebra (for safe queries). Neither can express aggregates, transitive closure, or arithmetic over relation-wide values.

Domain Relational Calculus (DRC)

Basic Selection — Sailor Names with Rating > 7

DRC Expression:

$$\{ \langle sn \rangle \mid \exists sid \exists r \exists a (Sailors(sid, sn, r, a) \wedge r > 7) \}$$

Variables: sid=SID, sn=SNAME, r=RATING, a=AGE

Read: Return sname (sn) where $\exists sid, r, a$ such that the tuple (sid,sn,r,a) is in Sailors AND $r > 7$

SQL Equivalent:

```
SELECT sname FROM Sailors WHERE rating > 7;
```

Result:

SNAME
Bob
Eve
Alice

Solution: DRC variables map one-to-one with attributes. $\exists$ is used for attributes not in the result set (sid, r, a still appear as existential variables to bind the tuple). Only sn is free/returned.

Note: Convention: use short mnemonic variable names matching column abbreviations (sid, sn, r, a) for readability.

Multi-Attribute Projection — Sailor ID, Name, and Rating**DRC Expression:**
$$\{ \langle \text{sid}, \text{sn}, \text{r} \rangle \mid \exists a (\text{Sailors}(\text{sid}, \text{sn}, \text{r}, a)) \}$$

Variables: sid=SID, sn=SNAME, r=RATING, a=AGE (existential — not returned)

Read: Return $\langle \text{sid}, \text{sn}, \text{r} \rangle$ for every $(\text{sid}, \text{sn}, \text{r}, a)$ tuple in Sailors

SQL Equivalent:

SELECT sid, sname, rating FROM Sailors;

Result:

SID	SNAME	RATING
1	Alice	8
2	Bob	10
3	Charlie	5
4	Diana	7
5	Eve	9
6	Frank	6

Solution: Projection in DRC: only desired attributes appear in the result tuple $\langle \dots \rangle$. Unneeded attributes (a=AGE here) are bound by $\exists$ to complete the relational atom but are not returned.

Note: The arity of the relational atom Sailors (sid,sn,r,a) must match the schema exactly: 4 variables for 4 columns.

Compound Predicate — Sailors with Rating ≥ 8 AND Age < 55 **DRC Expression:**
$$\{ \langle \text{sn}, \text{r}, \text{a} \rangle \mid \exists \text{sid} (\text{Sailors}(\text{sid}, \text{sn}, \text{r}, \text{a}) \wedge \text{r} \geq 8 \wedge \text{a} < 55) \}$$

Read: Return $\langle \text{sname}, \text{rating}, \text{age} \rangle$ where rating ≥ 8 AND age < 55

SQL Equivalent:

SELECT sname, rating, age FROM Sailors WHERE rating >= 8 AND age < 55;

Result:

SNAME	RATING	AGE
Alice	8	45.0
Eve	9	51.0

Solution: Multiple predicates on domain variables: $r \geq 8$ and $a < 55$ are appended with $\wedge$. Both must hold. Bob(63.5) is excluded by age. Only Alice and Eve satisfy both conditions.

Note: Domain variable a represents the AGE attribute value; predicates on it work the same as SQL WHERE conditions.

Existential Join — Names of Sailors Who Reserved Boat 101**DRC Expression:**

$$\{ \langle sn \rangle \mid \exists sid \exists r \exists a \exists bid \exists rd \\ (Sailors(sid, sn, r, a) \wedge \\ Reserves(sid, bid, rd) \wedge bid = 101) \}$$

Variables: $bid=BID$ in Reserves, $rd=RDATE$

Read: Sailor names joined with Reserves where bid equals 101

SQL Equivalent:

SELECT DISTINCT s.sname FROM Sailors s
 JOIN Reserves r ON s.sid=r.sid
 WHERE r.bid=101;

Result:

SNAME
Alice
Bob

SNAME

Diana

Solution: The join is implicit in DRC: sharing the variable sid between Sailors(sid,...) and Reserves(sid,...) forces the sid values to match — this is a natural join on sid.

Note: DRC join: use the SAME variable name in corresponding positions of two relational atoms. Different names would be a cross product.

Three-Table Join — Sailor Names and Boat Names for All Reservations**DRC Expression:**

$$\{ \langle sn, bn \rangle \mid \exists sid \exists r \exists a \exists bid \exists rd \exists bc \exists bt \\ (Sailors(sid, sn, r, a) \wedge \\ Reserves(sid, bid, rd) \wedge \\ Boats(bid, bn, bc, bt)) \}$$

Variables: bn=BNAME, bc=COLOR, bt=BOAT_TYPE

SQL Equivalent:

```
SELECT DISTINCT s.sname, b.bname FROM Sailors s
JOIN Reserves r ON s.sid=r.sid
JOIN Boats b ON r.bid=b.bid;
```

Result:

SNAME	BNAME
Alice	Nautilus
Alice	Phoenix
Bob	Nautilus
Bob	Phoenix
Charlie	Emerald

SNAME	BNAME
Diana	Nautilus
Eve	Titan
Frank	Emerald

Solution: Three-way join: sid is shared between Sailors and Reserves; bid is shared between Reserves and Boats. This chains the three relations. All 8 reservation pairs are returned.

Note: In DRC, every join is expressed by sharing variable names across relational atoms. No explicit JOIN keyword exists.

Selection with String Condition — Green Boats Only

DRC Expression:

$$\{ \langle \text{bid}, \text{bn} \rangle \mid \exists \text{bc} \exists \text{bt} \\ (\text{Boats}(\text{bid}, \text{bn}, \text{bc}, \text{bt}) \wedge \text{bc} = \text{'Green'}) \}$$

Variables: bid=BID, bn=BNAME, bc=COLOR, bt=BOAT_TYPE

SQL Equivalent:

SELECT bid, bname FROM Boats WHERE color='Green';

Result:

BID	BNAME
102	Phoenix
104	Emerald

Solution: String equality in DRC: the domain variable bc is bound to 'Green' by the predicate $\text{bc} = \text{'Green'}$. DRC predicates support all comparison operators: $=, \neq, <, >, \leq, \geq$.

Note: $bc = \text{'Green'}$ is a constant comparison. The variable bc itself is not returned (it's existential), but it constrains which Boats tuples are in scope.

Negation — Boats Never Reserved

DRC Expression:

$$\{ \langle bid, bn \rangle \mid \exists bc \exists bt (\text{Boats}(bid, bn, bc, bt) \wedge \neg \exists sid2 \exists rd (\text{Reserves}(sid2, bid, rd))) \}$$

Read: Boats (bid , $bname$) for which no reservation tuple exists

SQL Equivalent:

```
SELECT bid, bname FROM Boats
WHERE bid NOT IN (SELECT DISTINCT bid FROM Reserves);
```

Result:

BID	BNAME
—	(empty — all boats have been reserved)

Solution: Negation in DRC: $\neg \exists sid2 \exists rd (\text{Reserves}(sid2, bid, rd))$ checks that no reservation tuple contains this bid . Since all 4 boats appear in Reserves, the result is empty.

Note: The bid in the Reserves atom is the SAME variable as in the Boats atom — this is the join/correlation. $sid2$ and rd are fresh existential variables specific to Reserves.

Universal Quantifier — Boats Reserved by Every Sailor (Relational Division in DRC)

DRC Expression:

$$\{ \langle bid, bn \rangle \mid \exists bc \exists bt (\text{Boats}(bid, bn, bc, bt) \wedge \forall sid2 (\exists sn \exists r \exists a (\text{Sailors}(sid2, sn, r, a)) \Rightarrow \exists rd (\text{Reserves}(sid2, bid, rd)))) \}$$

Read: Boats such that for every sailor sid2, a reservation exists for this bid

SQL Equivalent:

```
SELECT b.bid, b.bname FROM Boats b
WHERE NOT EXISTS (
  SELECT s.sid FROM Sailors s
  WHERE NOT EXISTS (
    SELECT 1 FROM Reserves r
    WHERE r.sid=s.sid AND r.bid=b.bid
  )
);
```

Result:

BID	BNAME
—	(empty — no boat reserved by all 6 sailors)

Solution: Relational division in DRC: $\forall \text{sid2}$ quantifies over all sailor IDs, and the implication requires a matching Reserves tuple for each. The complex nesting is the DRC cost for expressing 'for all'. No boat qualifies here.

Note: $\forall x (P(x) \Rightarrow Q(x))$ is always equivalent to $\neg \exists x (P(x) \wedge \neg Q(x))$. Both forms appear in exams.

Comparison Between Relations — Sailors Younger Than Bob

DRC Expression:

```
{ <sn, a> |  $\exists \text{sid} \exists r$ 
  (Sailors(sid, sn, r, a)  $\wedge$ 
     $\exists a_{\text{bob}}$  (Sailors(2, 'Bob', 10, a_bob)  $\wedge$  a < a_bob)) }
```

Read: Sailors whose age is less than Bob's age (63.5)

SQL Equivalent:

```
SELECT sname, age FROM Sailors
WHERE age < (SELECT age FROM Sailors WHERE sname='Bob');
```

Result:

SNAME	AGE
Alice	45.0
Charlie	35.0
Diana	42.0
Eve	51.0
Frank	38.5

Solution: Inter-row comparison in DRC: a second occurrence of the Sailors atom with sid=2 binds a_bob to Bob's age. The predicate $a < a_bob$ compares domain variables across two instances of the same relation.

Note: Using two instances of the same relation with different variable sets is the DRC way to express a self-join or scalar subquery.

DRC for INTERSECT — Sailors Who Reserved Both Boat 101 and Boat 102**DRC Expression:**

$$\{ \langle sn \rangle \mid \exists sid \exists r \exists a \exists rd1 \exists rd2 \\ (Sailors(sid, sn, r, a) \wedge \\ Reserves(sid, 101, rd1) \wedge \\ Reserves(sid, 102, rd2)) \}$$

Read: Sailors with a reservation for bid=101 AND bid=102

SQL Equivalent:

```
SELECT sname FROM Sailors s
WHERE EXISTS (SELECT 1 FROM Reserves WHERE sid=s.sid AND bid=101)
AND EXISTS (SELECT 1 FROM Reserves WHERE sid=s.sid AND bid=102);
```

Result:

SNAME
Alice
Bob

Solution: DRC uses two Reserves atoms with the same sid but constants 101 and 102 for bid. Two independent fresh variables rd1 and rd2 bind the rdate of each reservation. Alice and Bob reserved both boats.

Note: Using a constant directly in the relational atom (Reserves(sid, 101, rd1)) is valid DRC — it constrains the bid domain variable to that specific value.

DRC with Inequality — Boats NOT of Type 'Sailboat'**DRC Expression:**

$$\{ \langle \text{bid}, \text{bn}, \text{bt} \rangle \mid \exists \text{bc} \\ (\text{Boats}(\text{bid}, \text{bn}, \text{bc}, \text{bt}) \wedge \text{bt} \neq \text{'Sailboat'}) \}$$

Read: Boats whose boat_type is not 'Sailboat'

SQL Equivalent:

```
SELECT bid, bname, boat_type FROM Boats WHERE boat_type <> 'Sailboat';
```

Result:

BID	BNAME	BOAT_TYPE
102	Phoenix	Yacht
103	Titan	Speedboat

Solution: Inequality ($\neq$) in DRC predicates: $\text{bt} \neq \text{'Sailboat'}$ excludes tuples where the boat_type domain variable equals 'Sailboat'. Maps to $\langle \rangle$ in SQL.

Note: DRC supports: $=$, $\neq$ ($\langle \rangle$), $<$, $>$, $\leq$, $\geq$ as predicate operators. These compare domain variables with constants or with other domain variables.

DRC Implication — Sailors Where Every Boat They Reserved is Blue or Red**DRC Expression:**

$$\{ \langle \text{sid}, \text{sn} \rangle \mid \exists r \exists a (\text{Sailors}(\text{sid}, \text{sn}, r, a) \wedge \\ \forall \text{bid2} \forall \text{rd2} (\text{Reserves}(\text{sid}, \text{bid2}, \text{rd2}) \\ \Rightarrow \exists \text{bn} \exists \text{bt} (\text{Boats}(\text{bid2}, \text{bn}, \text{bc}, \text{bt}) \\ \wedge (\text{bc} = \text{'Blue'} \vee \text{bc} = \text{'Red'})))) \}$$

Read: Sailors where all reserved boats have color Blue or Red

SQL Equivalent:

```
SELECT s.sid, s.sname FROM Sailors s
WHERE NOT EXISTS (
  SELECT 1 FROM Reserves r JOIN Boats b ON r.bid=b.bid
  WHERE r.sid=s.sid AND b.color NOT IN ('Blue','Red')
);
```

Result:

SID	SNAME
4	Diana
5	Eve

Solution: Universal implication: for every (bid2,rd2) pair in Reserves for this sailor, the boat must be Blue or Red. Alice reserved Nautilus(Red) AND Phoenix(Green) — Green is not Blue/Red → Alice EXCLUDED. Bob same. Charlie reserved Emerald(Green) → EXCLUDED. Frank reserved Emerald(Green) → EXCLUDED. Only Diana(Nautilus=Red) and Eve(Titan=Blue) qualify.

Note: Always trace through data manually to verify universal/implication queries. The result depends on each sailor's complete reservation history.

DRC — Rename / Alias Pattern for Two Instances of Same Relation**DRC Expression:**

$$\{ \langle sn1, sn2 \rangle \mid \exists sid1 \exists r1 \exists a1 \exists sid2 \exists r2 \exists a2 \\ (Sailors(sid1, sn1, r1, a1) \wedge \\ Sailors(sid2, sn2, r2, a2) \wedge \\ r1 = r2 \wedge sid1 < sid2) \}$$

Read: Pairs of distinct sailors with equal ratings

SQL Equivalent:

```
SELECT s1.sname, s2.sname FROM Sailors s1, Sailors s2
WHERE s1.rating = s2.rating AND s1.sid < s2.sid;
```

Result:

SNAME_1	SNAME_2
—	(no two sailors share the same rating in sample data)

Solution: Two instances of Sailors use completely separate variable sets (sid1,sn1,r1,a1) and (sid2,sn2,r2,a2). Shared rating: r1=r2. sid1<sid2 avoids symmetric duplicates. No rating matches in this dataset.

Note: The two-instance DRC self-join pattern is the standard way to express queries comparing two rows from the same table.

Competitive — DRC for Sailors Who Reserved at Least Two Different Boats**DRC Expression:**

$$\{ \langle sid, sn \rangle \mid \exists r \exists a \\ (Sailors(sid, sn, r, a) \wedge \\ \exists bid1 \exists rd1 \exists bid2 \exists rd2 \\ (Reserves(sid, bid1, rd1) \wedge \\ Reserves(sid, bid2, rd2) \wedge \\ bid1 \neq bid2)) \}$$

Read: Sailors with $\exists$ two distinct boat IDs in their reservations

SQL Equivalent:

```
SELECT s.sid, s.sname FROM Sailors s
WHERE (SELECT COUNT(DISTINCT bid) FROM Reserves r WHERE
r.sid=s.sid) >= 2;
```

Result:

SID	SNAME
1	Alice
2	Bob

Solution: Two instances of Reserves (with $\text{bid}_1 \neq \text{bid}_2$) express 'at least 2 distinct boats'. The inequality $\text{bid}_1 \neq \text{bid}_2$ is the key — without it, the same boat would satisfy both existentials. Alice reserved boats 101 and 102; Bob reserved 101 and 102.

Note: DRC has no COUNT. 'At least k' is expressed with k separate existential instances and pairwise inequality constraints.

Competitive — DRC Equivalent of GROUP BY / HAVING (Boats Reserved More Than Once)

DRC Expression:

$$\{ \langle \text{bid}, \text{bn} \rangle \mid \exists \text{bc} \exists \text{bt} (\text{Boats}(\text{bid}, \text{bn}, \text{bc}, \text{bt}) \wedge \\ \exists \text{sid}_1 \exists \text{rd}_1 \exists \text{sid}_2 \exists \text{rd}_2 \\ (\text{Reserves}(\text{sid}_1, \text{bid}, \text{rd}_1) \wedge \\ \text{Reserves}(\text{sid}_2, \text{bid}, \text{rd}_2) \wedge \\ (\text{sid}_1 \neq \text{sid}_2 \vee \text{rd}_1 \neq \text{rd}_2))) \}$$

Read: Boats with at least 2 distinct reservation tuples

SQL Equivalent:

```
SELECT b.bid, b.bname FROM Boats b
JOIN Reserves r ON b.bid=r.bid
```


GROUP BY b.bid, b.bname

HAVING COUNT(*) >= 2;

Result:

BID	BNAME
101	Nautilus
102	Phoenix
104	Emerald

Solution: Two Reserves instances with $(sid1 \neq sid2 \vee rd1 \neq rd2)$ ensure two distinct tuples. Nautilus was reserved by Alice, Bob, Diana (3 times); Phoenix by Alice and Bob (2 times); Emerald by Charlie and Frank (2 times). Titan (103) only once — excluded.

Note: DRC does not directly support aggregate functions such as COUNT. Conditions like $COUNT \geq 2$ are represented using multiple existential quantifiers with pairwise inequality to ensure distinct tuples. Higher counts require additional existential variables and inequality conditions.

Competitive — DRC with Arithmetic Comparison Between Domain Variables

DRC Expression:

$$\{ \langle sn1, r1, sn2, r2 \rangle \mid \\ \exists sid1 \exists a1 \exists sid2 \exists a2 \\ (Sailors(sid1, sn1, r1, a1) \wedge \\ Sailors(sid2, sn2, r2, a2) \wedge \\ r1 - r2 = 3 \wedge sid1 \neq sid2) \}$$

Read: Pairs of sailors where their ratings differ by exactly 3

SQL Equivalent:

```
SELECT s1.sname, s1.rating, s2.sname, s2.rating
FROM Sailors s1, Sailors s2
```

WHERE ABS(s1.rating - s2.rating) = 3 AND s1.sid <> s2.sid;

Result:

SNAME_1	RATING_1	SNAME_2	RATING_2
Alice	8	Charlie	5
Bob	10	Diana	7
Eve	9	Frank	6

Solution: Arithmetic difference between domain variables: $r1 - r2 = 3$.

DRC allows arithmetic expressions in predicates. Both orderings are returned ($sid1 \neq sid2$ but not $sid1 < sid2$), giving symmetric pairs.

Note: DRC supports arithmetic predicates on domain variables. The condition $r1 - r2 = 3$ is directional and differs from $|r1 - r2| = 3$. To retrieve symmetric rating pairs, both directions must be considered explicitly or handled using $ABS(r1-r2)=3$ in SQL.

Competitive — DRC Safety Analysis**DRC Expression:**

-- UNSAFE: $\{ \langle x, y \rangle \mid x > y \}$

No relational atom restricts x or y — infinite domain.

-- UNSAFE: $\{ \langle sid, sn \rangle \mid \neg \text{Sailors}(sid, sn, r, a) \}$

Variables not domain-restricted to any base relation.

-- SAFE: $\{ \langle sn \rangle \mid \exists sid \exists r \exists a (\text{Sailors}(sid, sn, r, a) \wedge r > 7) \}$

sn is restricted by Sailors atom.

SQL Equivalent:

SELECT sname FROM Sailors WHERE rating > 7;

Result:

EXPRESSION TYPE	SAFE?	REASON
$\{ \langle x,y \rangle \mid x > y \}$	NO	No relational atom — x,y range over infinite domain
$\{ \langle \text{sid} \rangle \mid \neg \text{Sailors}(\text{sid}, \dots) \}$	NO	Negation without positive base atom — infinite complement
$\{ \langle \text{sn} \rangle \mid \exists \text{sid} \exists r \exists a (\text{Sailors}(\text{sid}, \text{sn}, r, a) \wedge r > 7) \}$	YES	sn restricted by Sailors atom
$\{ \langle \text{bid} \rangle \mid \exists bc (\text{Boats}(\dots) \wedge bc = \text{'Red'}) \}$	YES	bid restricted by Boats atom

Solution: Safety in DRC: an expression is safe if every free variable appears in the result AND is restricted to the active domain via a positive relational atom. Negation alone cannot restrict a variable — it needs a companion positive atom.

Note: Rule of thumb: for every free variable x_i in $\langle x_1, \dots, x_n \rangle$, there must exist at least one positive relational atom $R(\dots x_i \dots)$ in the predicate.

Competitive — TRC vs DRC Comparison on the Same Query**DRC Expression:**

Query: Names of sailors who reserved a green boat

TRC: $\{ t.\text{tname} \mid \text{Sailors}(t) \wedge \exists r (\text{Reserves}(r) \wedge r.\text{sid} = t.\text{sid} \wedge \exists b (\text{Boats}(b) \wedge b.\text{bid} = r.\text{bid} \wedge b.\text{color} = \text{'Green'})) \}$

DRC: $\{ \langle \text{sn} \rangle \mid \exists \text{sid} \exists r \exists a \exists \text{bid} \exists r_d \exists b_n \exists b_t (\text{Sailors}(\text{sid}, \text{sn}, r, a) \wedge \text{Reserves}(\text{sid}, \text{bid}, r_d) \wedge \text{Boats}(\text{bid}, b_n, \text{'Green'}, b_t)) \}$

Note: In DRC, 'Green' can be a constant in the Boats atom position.

SQL Equivalent:

```
SELECT DISTINCT s.sname FROM Sailors s
JOIN Reserves r ON s.sid=r.sid
JOIN Boats b ON r.bid=b.bid
WHERE b.color='Green';
```

Result:

SNAME
Alice
Bob
Charlie
Frank

Solution: Both TRC and DRC are equivalent in expressive power to relational algebra (for safe queries). TRC uses tuple variables (t.attr); DRC uses domain variables (one per column). TRC is often more concise; DRC explicitly lists every attribute.

Note: Exam tip: TRC and DRC are both non-procedural. Relational algebra is procedural. All three are equivalent in expressiveness for safe queries.

Competitive — DRC Query for Maximum-Rated Sailor Per Boat Type**DRC Expression:**

$$\{ \langle bt, sn, r \rangle \mid$$

$$\exists sid \exists a \exists bid \exists rd \exists bn \exists bc$$

$$(Sailors(sid, sn, r, a) \wedge$$

$$Reserves(sid, bid, rd) \wedge$$

$$Boats(bid, bn, bc, bt) \wedge$$

$$\neg \exists sid2 \exists sn2 \exists r2 \exists a2 \exists bid2 \exists rd2$$

$$(Sailors(sid2, sn2, r2, a2) \wedge Reserves(sid2, bid2, rd2) \wedge$$

Boats(bid2,bn,bc,bt) $\wedge$ r2 > r) } }

Read: For each boat type, the sailor with the highest rating who reserved that type

SQL Equivalent:

WITH ranked AS (

SELECT b.boat_type, s.sname, s.rating,

ROW_NUMBER() OVER (PARTITION BY b.boat_type ORDER BY s.rating
DESC) rn

FROM Sailors s JOIN Reserves r ON s.sid=r.sid

JOIN Boats b ON r.bid=b.bid

)

SELECT boat_type, sname, rating FROM ranked WHERE rn=1;

Result:

BOAT_TYPE	SNAME	RATING
Sailboat	Bob	10
Speedboat	Eve	9
Yacht	Alice	8

Solution: MAX-per-group in DRC: $\neg \exists$ another sailor with higher rating in same boat type. The inner negation checks no sailor with $r_2 > r$ exists for the same bt. DRC achieves this through nested quantifiers rather than SQL's window functions.

Note: This query is complex in DRC/TRC because neither has GROUP BY. In practice, SQL's ROW_NUMBER() PARTITION BY is vastly more readable for this pattern.

Competitive — Summary: TRC, DRC, and Relational Algebra Equivalence

DRC Expression:

Fundamental Theorem (Codd, 1972):

Safe TRC $\equiv$ Safe DRC $\equiv$ Relational Algebra (RA)

Mapping:

RA Selection $\sigma_P(R) \leftrightarrow \text{TRC: } \{ t \mid R(t) \wedge P(t) \}$

$\leftrightarrow \text{DRC: } \{ \langle x, \dots \rangle \mid R(x, \dots) \wedge P(x, \dots) \}$

RA Projection $\pi_A(R) \leftrightarrow \text{TRC: } \{ t.A \mid R(t) \}$

$\leftrightarrow \text{DRC: } \{ \langle a \rangle \mid \exists \dots (R(\dots, a, \dots)) \}$

RA Join $R \bowtie S \leftrightarrow \text{TRC: } \{ t, u \mid R(t) \wedge S(u) \wedge t.k = u.k \}$

$\leftrightarrow \text{DRC: } \{ \langle x, \dots, y, \dots \rangle \mid R(k, x, \dots) \wedge S(k, y, \dots) \}$

RA Union $R \cup S \leftrightarrow \text{TRC: } \{ t \mid R(t) \vee S(t) \}$

RA Difference $R - S \leftrightarrow \text{TRC: } \{ t \mid R(t) \wedge \neg \exists u (S(u) \wedge u=t) \}$

RA Division $R \div S \leftrightarrow \text{TRC: } \forall \text{ with } \Rightarrow \text{ (universal quantifier)}$

SQL Equivalent:

-- Relational Algebra (procedural) tells HOW to compute

-- TRC/DRC (declarative) tells WHAT to retrieve

-- SQL is based on TRC (FROM=range, WHERE=predicate, SELECT=result tuple)

Result:

Feature	Relational Algebra	TRC	DRC
Style	Procedural	Declarative	Declarative
Variable type	Sets of tuples	Tuple variables	Domain (attribute) variables
Joins	Explicit $\bowtie$	Implicit via shared tuple var	Implicit via shared domain var

Feature	Relational Algebra	TRC	DRC
Aggregation	Extended RA (G)	Not supported (pure)	Not supported (pure)
Safety	Always safe	Requires safety condition	Requires safety condition
SQL basis	Core operations	FROM/WHERE/SELECT maps to TRC	Attribute-level reasoning

Solution: Codd's theorem (1972): safe TRC $\equiv$ safe DRC $\equiv$ relational algebra in expressive power. SQL is a practical language derived from TRC with extensions (aggregation, ORDER BY, recursion). All standard SQL queries can be expressed in safe TRC and vice versa.

Note: Exam tip: The three formalisms are equivalent ONLY for safe queries. RA is always safe by definition. TRC/DRC require explicit safety verification.

TRC Expression Patterns

Pattern	TRC Syntax	SQL Equivalent
Selection	$\{ t \mid R(t) \wedge t.A = v \}$	SELECT * FROM R WHERE A = v
Projection	$\{ t.A, t.B \mid R(t) \}$	SELECT DISTINCT A, B FROM R
Join	$\{ t.A, u.B \mid R(t) \wedge S(u) \wedge t.k = u.k \}$	SELECT R.A, S.B FROM R JOIN S ON R.k=S.k
Union	$\{ t \mid R(t) \vee S(t) \}$	SELECT * FROM R UNION SELECT * FROM S
Difference	$\{ t \mid R(t) \wedge \neg \exists u (S(u) \wedge u.k = t.k) \}$	SELECT * FROM R WHERE k NOT IN (SELECT k FROM S)

Pattern	TRC Syntax	SQL Equivalent
EXISTS	$\{ t \mid R(t) \wedge \exists u(S(u) \wedge u.k=t.k) \}$	SELECT * FROM R WHERE EXISTS (SELECT 1 FROM S WHERE S.k=R.k)
NOT EXISTS	$\{ t \mid R(t) \wedge \neg \exists u(S(u) \wedge u.k=t.k) \}$	SELECT * FROM R WHERE NOT EXISTS (SELECT 1 FROM S WHERE S.k=R.k)
FOR ALL (division)	$\{ t \mid R(t) \wedge \forall u(S(u) \Rightarrow \exists v(T(v) \wedge \dots)) \}$	Double NOT EXISTS
MAX	$\{ t \mid R(t) \wedge \neg \exists s(R(s) \wedge s.A > t.A) \}$	SELECT * FROM R WHERE A=(SELECT MAX(A) FROM R)

DRC Expression Patterns

Pattern	DRC Syntax	Key Point
Selection	$\{ \langle x_1, \dots \rangle \mid R(x_1, \dots) \wedge x_i = v \}$	All attributes listed; result variables are free
Projection	$\{ \langle x_a, x_b \rangle \mid \exists x_c \dots x_n (R(x_1, \dots, x_n)) \}$	Non-result attributes become $\exists$ bound
Join	$\{ \langle x, y \rangle \mid R(x, \dots, k, \dots) \wedge S(k, \dots, y, \dots) \}$	Shared variable k is the join attribute
Negation	$\{ \langle x \rangle \mid R(x, \dots) \wedge \neg \exists y \dots (S(x, \dots, y, \dots)) \}$	x must appear in positive atom; $\neg$ on inner $\exists$
Universal	$\{ \langle x \rangle \mid R(x, \dots) \wedge \forall y (S(y, \dots) \Rightarrow \dots) \}$	$\forall y(A \Rightarrow B) \equiv \neg \exists y(A \wedge \neg B)$

Pattern	DRC Syntax	Key Point
Constant in atom	Boats(bid, bn, 'Red', bt)	Can use constant directly in atom position
Same relation twice	$R(x1,...) \wedge R(x2,...)$	Use completely separate variable sets

Quantifier Laws (De Morgan for Quantifiers)

Law	Expression	Equivalent
De Morgan 1	$\neg \exists x P(x)$	$\forall x \neg P(x)$
De Morgan 2	$\neg \forall x P(x)$	$\exists x \neg P(x)$
Implication	$\forall x (A(x) \Rightarrow B(x))$	$\neg \exists x (A(x) \wedge \neg B(x))$
Conjunction	$\exists x (A(x) \wedge B(x))$	Find x satisfying both A and B simultaneously
Disjunction	$\exists x (A(x) \vee B(x))$	Find x satisfying at least one of A or B
Nested $\exists$	$\exists x \exists y P(x,y)$	$\exists y \exists x P(x,y)$ — order of $\exists$ doesn't matter
Mixed $\forall \exists$	$\forall x \exists y P(x,y)$	NOT same as $\exists y \forall x P(x,y)$ — order matters!

Case Study 5: Functional Dependencies and Normalization

Setup — Schema & Data

```
CREATE TABLE Student(sid INT PRIMARY KEY, sname VARCHAR(40) NOT
NULL, dept VARCHAR(20));
CREATE TABLE Course(cid INT PRIMARY KEY, cname VARCHAR(50),
credits INT);
CREATE TABLE TeachesInfo(cid INT, prof_name VARCHAR(40), prof_office
VARCHAR(20), PRIMARY KEY(cid,prof_name));
CREATE TABLE StudentCourse_Norm(sid INT, sname VARCHAR(40), dept
VARCHAR(20), cid INT,
cname VARCHAR(50), credits INT, prof_name VARCHAR(40), prof_office
VARCHAR(20),
PRIMARY KEY(sid,cid));
```

SID	SNAME	DEPT
1	Alice	CSE
2	Bob	ECE
3	Carol	CSE

CID	CNAME	CREDITS
301	DBMS	4
302	Networks	4
303	OS	3

CID	PROF_NAME	PROF_OFFICE
301	Dr. Kumar	A101
302	Dr. Patel	B205
303	Dr. Singh	A201

Q71 Identify Functional Dependencies

Query:

```
SELECT 'FD: sid -> sname, dept' AS fd1,
'FD: cid -> cname, credits' AS fd2,
'FD: cid -> prof_name, prof_office' AS fd3;
```

Output:

FD1	FD2	FD3
$\text{sid} \rightarrow \text{sname, dept}$	$\text{cid} \rightarrow \text{cname, credits}$	$\text{cid} \rightarrow \text{prof_name, prof_office}$

Solution: Student attributes depend on sid; course attributes on cid; professor on cid.

Q72 Decompose to 1NF — Eliminate Non-Key Dependency on Part of Composite Key

Query:

```
SELECT 'StudentCourse_Norm(sid,cid) with FK to Student and Course;
TeachesInfo(cid,prof) separate' AS normalized;
```

Output:

NORMALIZED
Separate Student(sid,sname,dept), Course(cid,cname,credits), and TeachesInfo relations.

Solution: 1NF: Separate Student, Course, and TeachesInfo relations.

Q73 Detect 2NF Violation — Partial Dependency on Composite Key

Query:

```
SELECT 'StudentCourse(sid,cid)->sname,dept depends only on sid (not full
key)' AS violation;
```

Output:

VIOLATION
StudentCourse(sid,cid)->sname,dept depends only on sid (not full key)

Solution: 2NF: Non-prime attributes must depend on entire composite key.

Q74 Decompose to 2NF — Remove Partial Dependency

Query:

```
SELECT 'Extract Student relation with FD sid->sname,dept' AS step1,
'Keep Enroll(sid,cid) junction; remove non-key attributes' AS step2;
```

Output:

STEP1	STEP2
Extract Student with FD sid->sname,dept	Keep Enroll(sid,cid); remove non-key attributes

Solution: 2NF: Place sid-dependent attributes in Student; sid,cid pair in Enrollment.

Q75 Identify Transitive Dependency — 3NF Violation

Query:

```
SELECT 'FD chain: cid -> prof_name -> prof_office' AS transitive,
       'FD: cid -> prof_name (determinant) -> prof_office (non-PK attribute)'
AS violation;
```

Output:

TRANSITIVE	VIOLATION
cid → prof_name → prof_office	cid → prof_name (determinant) → prof_office (non-PK attribute)

Solution: 3NF: Transitive cid→prof→office violates; decompose into separate Teaches relation.

Q76 Normalize to 3NF — Remove Transitive Dependencies

Query:

```
SELECT 'Course(cid,cname,credits), Teaches(cid,prof),
       Professor(prof,office)' AS schema3nf;
```

Output:

SCHEMA_3NF
Course(cid,cname,credits), Teaches(cid,prof), Professor(prof,office)

Solution: 3NF: All non-key attributes depend directly on primary key.

Q77 Test BCNF — Every Determinant is a Superkey

Query:

```
SELECT 'Decomposed schema: all FDs have LHS as candidate key or
       superkey' AS bcnf_check;
```

Output:

BCNF_CHECK
All FDs have LHS as candidate key or superkey

Solution: BCNF: Every determinant must be a superkey (stricter than 3NF).

Q78 Minimal Cover — Redundant FD Elimination

Query:

```
SELECT 'Given FDs: A->BC, B->C, AB->D' AS fds_list,
```

'Minimal cover: $A \rightarrow BC$, $A \rightarrow D$ ($B \rightarrow C$ subsumed by transitivity)' AS
minimal_cover;

Output:

FDS_LIST	MINIMAL_COVER
$A \rightarrow BC$, $B \rightarrow C$, $AB \rightarrow D$	Minimal cover: $A \rightarrow BC$, $A \rightarrow D$ ($B \rightarrow C$ redundant by transitivity)

Solution: FD minimization: remove redundant dependencies; competitive exam algorithm analysis.

Q79 BCNF Decomposition — Lossless Join Verification

Query:

SELECT 'R(A,B,C) with FDs $A \rightarrow B$, $B \rightarrow A$; $B \rightarrow C$ violates BCNF' AS problem,
'Decompose into R1(A,B) and R2(B,C); natural join recovers R losslessly'
AS solution;

Output:

PROBLEM	SOLUTION
R(A,B,C): $A \rightarrow B$, $B \rightarrow A$; $B \rightarrow C$ violates BCNF	Decompose R1(A,B), R2(B,C); lossless join recovers R

Solution: BCNF decomposition ensuring lossless join and dependency preservation.

Q80 Multivalued Dependencies (MVD) — 4NF Decomposition

Query:

SELECT 'R(sid,skill,language): $\text{sid} \twoheadrightarrow \text{skill}$ AND $\text{sid} \twoheadrightarrow \text{language}$ ' AS
mvd_analysis,
'Decompose: StudentSkills(sid,skill), StudentLanguages(sid,language)'
AS solution;

Output:

MVD_ANALYSIS	SOLUTION
$\text{sid} \twoheadrightarrow \text{skill}$ AND $\text{sid} \twoheadrightarrow \text{language}$ (independent)	StudentSkills(sid,skill) + StudentLanguages(sid,language)

Solution: 4NF: Eliminate non-product MVDs; highest practical normalization form.

Case Study 6: Transactions, ACID, and Concurrency Control

Units V

Setup — Schema & Data

```
CREATE TABLE BankAccount(acct_id INT PRIMARY KEY, holder_name
VARCHAR(40),
    balance DECIMAL(10,2) CHECK(balance>=0));
CREATE TABLE TransactionLog(txn_id INT PRIMARY KEY, acct_id INT,
    txn_type VARCHAR(10) CHECK(txn_type IN ('DEBIT','CREDIT')),
    amount DECIMAL(10,2) CHECK(amount>0), timestamp TIMESTAMP,
status VARCHAR(20));
CREATE TABLE LockTable(lock_id INT PRIMARY KEY, acct_id INT, txn_id
INT,
    lock_type VARCHAR(10), acquire_ts TIMESTAMP);
```

ACCT_ID	HOLDER_NAME	BALANCE
501	Alice Johnson	5000
502	Bob Smith	3500.5
503	Charlie Lee	2000

TXN_ID	ACCT_ID	TXN_TYPE	AMOUNT	TIMESTAMP	STATUS
1	501	DEBIT	500	2026-02-12 10:30:00	COMMITTED
2	502	CREDIT	500	2026-02-12 10:31:00	COMMITTED
3	501	CREDIT	1000	2026-02-12 10:32:00	COMMITTED

Q81 Atomicity — All-or-Nothing Transfer Transaction

Query:

```
BEGIN TRANSACTION;
    UPDATE BankAccount SET balance=balance-500 WHERE acct_id=501;
    UPDATE BankAccount SET balance=balance+500 WHERE acct_id=502;
COMMIT;
SELECT acct_id, holder_name, balance FROM BankAccount WHERE acct_id
IN(501,502);
```

Output:

ACCT_ID	HOLDER_NAME	BALANCE
501	Alice Johnson	4500
502	Bob Smith	4000.5

Solution: Atomicity: both updates succeed or both rollback; no partial transfers.

Q82 Consistency — Enforce Account Constraints via CHECK**Query:**

```
BEGIN;
UPDATE BankAccount SET balance=balance-10000 WHERE acct_id=501;
ROLLBACK;
SELECT acct_id, balance FROM BankAccount WHERE acct_id=501;
```

Output:

ACCT_ID	BALANCE
501	5000

Solution: Consistency: negative balance prevented; transaction rolled back to maintain invariants.

Q83 Isolation — Concurrent READ/WRITE with EXCLUSIVE Locks**Query:**

```
SELECT 'T1: Read acct 501 (balance=5000), acquire EXCLUSIVE lock' AS
t1_step1,
      'T2: blocked on EXCLUSIVE lock for acct 501' AS t2_wait,
      'T1: Write acct 501 (balance=4500), commit' AS t1_commit,
      'T2: Read acct 501 (balance=4500) after T1 commit' AS t2_read_clean;
```

Output:

T1_STEP1	T2_WAIT	T1_COMMIT	T2_READ
T1: acquire EXCLUSIVE lock	T2: blocked on lock	T1: write, commit	T2: reads clean value

Solution: Isolation: EXCLUSIVE lock prevents dirty reads; serializes conflicting transactions.

Q84 Durability — Committed Transactions Survive Crash

Query:

```
SELECT txn_id, acct_id, txn_type, amount, status
FROM TransactionLog
WHERE status='COMMITTED'
ORDER BY txn_id;
```

Output:

TXN_ID	ACCT_ID	TXN_TYPE	AMOUNT	STATUS
1	501	DEBIT	500	COMMITTED
2	502	CREDIT	500	COMMITTED
3	501	CREDIT	1000	COMMITTED

Solution: Durability: committed transactions in persistent log; survive media failure and restart.

Q85 Serializability — Conflict Detection in Schedules

Query:

```
SELECT 'Schedule: T1:R(A), T2:R(A), T1:W(A), T2:W(A)' AS
non_serializable,
'Conflicting: T1:W(A) with T2:R(A) and T2:W(A)' AS conflict,
'Cycle: T1->T2 but T2:R(A) before T1:W(A)' AS cycle;
```

Output:

NON_SERIAL	CONFLICT	CYCLE
T1:R(A),T2:R(A),T1:W(A),T2:W(A)	T1:W(A) conflicts T2:R(A) and T2:W(A)	Precedence cycle detected

Solution: Non-serializable: conflict graph contains cycle; violates serializability condition.

Q86 2PL — Two-Phase Locking Protocol Compliance

Query:

```
SELECT 'Phase 1 (Growing): T1 acquires locks on A, B, C sequentially' AS
growing,
'Phase 2 (Shrinking): T1 releases locks after commit in reverse order'
AS shrinking;
```


Output:

GROWING	SHRINKING
Phase 1: T1 acquires locks on A, B, C	Phase 2: T1 releases locks after commit

Solution: 2PL: lock acquisition then release; ensures conflict serializability.

Q87 Timestamp-Based Concurrency — Read-Write Ordering**Query:**

SELECT 'T1 (TS=100) reads X, T2 (TS=50) writes X' AS conflict,
 'T2.TS < T1.TS => write rejected (T1 must see T2 pre-write or abort T2)' AS resolution;

Output:

CONFLICT	RESOLUTION
T1(TS=100) reads X, T2(TS=50) writes X	T2.TS < T1.TS: write rejected; T2 aborted

Solution: Timestamp ordering: read-after-write ordering via TS comparison; ensures serializability.

Q88 Deadlock Detection via Wait-For Graph**Query:**

SELECT 'T1 waits for T2, T2 waits for T3, T3 waits for T1' AS wait_for_edge,
 'Cycle T1->T2->T3->T1 detected' AS detection,
 'Victim: T2 (youngest); rollback and retry' AS resolution;

Output:

WAIT_FOR_EDGE	DETECTION	RESOLUTION
T1→T2, T2→T3, T3→T1	Cycle T1→T2→T3→T1	Victim: T2 (youngest); rollback

Solution: Deadlock via wait-for cycle; cycle detection algorithm; victim selection for resolution.

Q89 WAL Recovery — REDO/UNDO with Checkpoint**Query:**

SELECT 'Crash at LSN=8000, LastCheckpoint=5000; T7(LSN=5500),
 T9(LSN=6200)' AS scenario,
 'REDO Phase: scan LSN 5001-8000; reapply all updates' AS redo,
 'UNDO Phase: scan backward; undo uncommitted T7, T9 via pre-images' AS undo;

Output:

SCENARIO	REDO	UNDO
Crash LSN=8000, chkpt=5000	REDO: reapply from LSN 5001	UNDO: reverse T7, T9

Solution: WAL recovery with checkpoint: REDO all forward; UNDO uncommitted backward; ensures ACID.

Q90 Phantom Read Prevention — SERIALIZABLE + Range Locks**Query:**

SELECT 'Isolation: SERIALIZABLE' AS level,
 'T1 reads WHERE balance>10000 (range lock)' AS t1,
 'T2 INSERT balance=15000 blocked by range lock' AS prevention;

Output:

LEVEL	T1_RANGE_LOCK	T2_BLOCKED
SERIALIZABLE	T1: range lock on balance > 10000	T2 INSERT blocked; phantom prevented

Solution: SERIALIZABLE + range locking: prevents phantom reads; highest isolation at performance cost.

Q91 Multi-Level Recovery — B+ Tree Integrity After Crash**Query:**

SELECT 'Crash during index insert: incomplete B+ node split' AS problem,
 'Recovery: detect via LSN; rebuild B+ tree; reapply from checkpoint' AS recovery,
 'Verify: index scan matches base table scan; ACID restored' AS verification;

Output:

PROBLEM	RECOVERY	VERIFICATION
Incomplete B+ split during crash	LSN-based redo/undo; rebuild index	Index scan matches table scan

Solution: Index recovery: ensure B+ tree consistency via LSN-based redo/undo; maintain structural integrity.

Case Study 7: Advanced Indexing, Storage, and Query Optimization

Units V

Setup — Schema & Data

PID	PNAME	CATEGORY	PRICE	STOCK	RESTOCK_DATE
2001	Laptop Pro Z15	Electronics	1500	120	2026-02-10
2002	USB-C Cable 2m	Electronics	25	500	2026-01-25
2003	Ergonomic Chair	Furniture	350	80	2026-02-01
2004	4K Monitor 32in	Electronics	450	60	2026-02-08
2005	Mechanical Keyboard RGB	Electronics	125	200	2026-02-03

IDX_NAME	TABLE_REF	SCANS	SEEKS	KEY_COMPARISONS
idx_prod_cat	Product	2000	5600	18000
idx_prod_price	Product	800	3200	9500
idx_sales_pid	SalesOrder	600	4100	12300

Q92 B+ Tree Index — Point Query Seek Operation

Query:

```
CREATE INDEX idx_product_pid ON Product(pid) USING BTREE;  
SELECT pid, pname FROM Product WHERE pid=2004;
```

Output:

PID	PNAME
2004	4K Monitor 32in

Solution: B+ tree point search: $O(\log N)$ via internal node traversal to leaf; single seek.

Q93 Range Query — B+ Tree Leaf Sibling Traversal

Query:

```
SELECT pid, pname, price FROM Product WHERE price BETWEEN 100 AND 500 ORDER BY price;
```

Output:

PID	PNAME	PRICE
2005	Mechanical Keyboard RGB	125
2003	Ergonomic Chair	350
2004	4K Monitor 32in	450

Solution: B+ tree range: leaf sibling pointers enable efficient sequential retrieval without re-traversal.

Q94 Hash Index — Equality-Only Lookups

Query:

```
CREATE INDEX idx_product_cat_hash ON Product USING HASH(category);
SELECT pid, pname FROM Product WHERE category='Electronics' ORDER BY pid;
```

Output:

PID	PNAME
2001	Laptop Pro Z15
2002	USB-C Cable 2m
2004	4K Monitor 32in
2005	Mechanical Keyboard RGB

Solution: Hash index: $O(1)$ average for equalities; no ordering support or range queries.

Q95 Composite Index — Multi-Column Filtering

Query:

```
CREATE INDEX idx_prod_composite ON Product(category,price);
SELECT pid, pname, price FROM Product
WHERE category='Electronics' AND price<500
ORDER BY price;
```

Output:

PID	PNAME	PRICE
2002	USB-C Cable 2m	25

PID	PNAME	PRICE
2005	Mechanical Keyboard RGB	125
2004	4K Monitor 32in	450

Solution: Composite index: leading column enables efficient filtering; trailing columns for further refinement.

Q96 Covering Index — Index-Only Scan

Query:

```
CREATE INDEX idx_prod_covering ON Product(category,price,pname)
INCLUDE (stock_qty);
SELECT pname, price, stock_qty FROM Product
WHERE category='Electronics' AND price<300;
```

Output:

PNAME	PRICE	STOCK_QTY
USB-C Cable 2m	25	500
Mechanical Keyboard RGB	125	200

Solution: Covering index: all columns in query available in index; avoids base table access.

Q97 Index Usage Statistics — Seek vs Scan Ratio

Query:

```
SELECT idx_name, scans, seeks,
       ROUND(seeks*100.0/(scans+seeks),1) AS seek_percentage
FROM IndexStats WHERE seeks>scans
ORDER BY seek_percentage DESC;
```

Output:

IDX_NAME	SCANS	SEEKS	SEEK_PERCENTAGE
idx_sales_pid	600	4100	87.2
idx_prod_price	800	3200	80
idx_prod_cat	2000	5600	73.7

Solution: High seek%: index well-optimized for point/range queries; justify maintenance overhead.

Q98 Cost-Based Optimizer — Multi-Index Join Plan

Query:

```
EXPLAIN ANALYZE
```

```
SELECT p.pid, p.pname, s.oid, s.qty_ordered
FROM Product p JOIN SalesOrder s ON p.pid=s.pid
WHERE p.category='Electronics' AND p.price<600
      AND s.order_date>'2026-02-12'
ORDER BY p.price, s.order_date DESC;
```

Output:

STEP	EST_COST	ACTUAL_COST	INDEX_USED
Filter: category=Electronics AND price<600	250ms	240ms	idx_prod_composite
Index seek SalesOrder.pid	120ms	118ms	idx_sales_pid
Nested loop join	150ms	148ms	inline
Sort (ORDER BY)	80ms	75ms	external

Solution: Optimizer: multi-index use for filter/join; nested loop with sort; competitive execution planning.

Q99 B+ Tree Insertion — Node Split and Rebalancing

Query:

```
SELECT 'Insert key 550 into B+ tree (order d=2: max 4 keys per node)' AS
operation,
      'Leaf overflow: (100,200,450,550,600) exceeds max 4' AS overflow,
      'Split: left (100,200,450), right (550,600); promote 500 to parent' AS
split_action;
```

Output:

OPERATION	OVERFLOW	SPLIT_ACTION
Insert key 550 (max 4 keys)	(100,200,450,550,600) overflows	Left (100,200,450) Right (550,600); promote 500

Solution: B+ tree insertion: splits maintain balance; promotes separators up; $O(\log N)$ height guaranteed.

Q100 Materialized View — Repeated Aggregation Optimization

Query:

```
CREATE MATERIALIZED VIEW mv_electronics_sales AS
SELECT p.category, p.pid, p.pname,
      COUNT(DISTINCT s.oid) AS order_count,
      SUM(s.qty_ordered) AS volume,
      SUM(s.total_price) AS revenue
```

```
FROM Product p LEFT JOIN SalesOrder s ON p.pid=s.pid
WHERE p.category='Electronics'
GROUP BY p.category, p.pid, p.pname;
```

```
SELECT category, pname, order_count, revenue
FROM mv_electronics_sales
ORDER BY revenue DESC;
```

Output:

CATEGORY	PNAME	ORDER_COUNT	REVENUE
Electronics	Laptop Pro Z15	2	4500
Electronics	4K Monitor 32in	1	1350
Electronics	USB-C Cable 2m	1	250
Electronics	Mechanical Keyboard RGB	0	0

Solution: Materialized view: avoids repeated join computation; faster aggregation queries; requires refresh.

Q101 Dynamic Index Selection — Workload Shift Analysis**Query:**

```
SELECT 'Old: 60% point-pid, 30% range-price, 10% full-scan' AS
old_workload,
      'New: 80% point-pid, 15% range category+price' AS new_workload,
      'Keep idx_sales_pid; promote idx_prod_composite; drop idx_prod_price
(redundant)' AS decision;
```

Output:

OLD_WORKLOAD	NEW_WORKLOAD	DECISION
60% pid, 30% price, 10% scan	80% pid, 15% cat+price	Keep idx_sales_pid; promote composite; drop price-only

Solution: Index tuning: adjust based on workload statistics and cost-benefit analysis.

Case Study 8: SQL Advanced Features — Views, Triggers, Stored Procedures

Units III-IV

Setup — Schema & Data

EID	ENAME	DEPT	SALARY	STATUS
301	Arjun	CSE	85000	ACTIVE
302	Bhavna	CSE	92000	ACTIVE
303	Chirag	ECE	78000	ACTIVE
304	Diana	Finance	70000	INACTIVE

DEPT	ANNUAL_BUDGET	YTD_SPENT
CSE	5000000	1800000
ECE	4500000	900000
Finance	3000000	500000

Q102 Non-Updatable View — Department Salary Analytics

Query:

```
CREATE VIEW EmployeeSalaryView AS
SELECT e.eid, e.ename, e.dept, e.salary, d.annual_budget,
       ROUND(100.0*e.salary/d.annual_budget,3) AS salary_pct_of_budget
FROM EmployeeRaw e JOIN DepartmentBudget d ON e.dept=d.dept
WHERE e.status='ACTIVE';
```

```
SELECT * FROM EmployeeSalaryView WHERE dept='CSE' ORDER BY salary
DESC;
```

Output:

EID	ENAME	DEPT	SALARY	ANNUAL_BUDGET	SALARY_PCT
302	Bhavna	CSE	92000	5000000	1.84
301	Arjun	CSE	85000	5000000	1.7

Solution: Non-updatable view with derived column; aggregate function prevents UPDATE.

Q103 AFTER Trigger — Audit Logging on Salary Changes**Query:**

```
CREATE TRIGGER trg_salary_change
AFTER UPDATE ON EmployeeRaw
FOR EACH ROW WHEN (NEW.salary <> OLD.salary)
BEGIN
    INSERT INTO AuditLog(eid, action, old_value, new_value, audit_ts)
    VALUES (NEW.eid, 'SALARY_CHANGE',
            CAST(OLD.salary AS VARCHAR), CAST(NEW.salary AS VARCHAR),
            CURRENT_TIMESTAMP);
END;

UPDATE EmployeeRaw SET salary=88000 WHERE eid=301;
SELECT action, old_value, new_value FROM AuditLog WHERE eid=301
ORDER BY log_id DESC LIMIT 1;
```

Output:

ACTION	OLD_VALUE	NEW_VALUE
SALARY_CHANGE	85000	88000

Solution: Row-level AFTER trigger: auto-logs salary changes for compliance audit.

Q104 BEFORE Trigger — Restrict Updates on Inactive Employees**Query:**

```
CREATE TRIGGER trg_prevent_inactive_update
BEFORE UPDATE ON EmployeeRaw
FOR EACH ROW WHEN (OLD.status='INACTIVE')
BEGIN
    RAISE EXCEPTION 'Cannot modify inactive employee';
END;
```

```
UPDATE EmployeeRaw SET salary=75000 WHERE eid=304;
```

Output:

ERROR
Cannot modify inactive employee record

Solution: BEFORE trigger enforces business rule: inactive employees immutable.

Q105 Aggregate View — Departmental KPIs

Query:

```
CREATE VIEW DeptAggregate AS
SELECT e.dept, COUNT(DISTINCT e.eid) AS emp_count,
       ROUND(AVG(e.salary),0) AS avg_salary,
       SUM(e.salary) AS total_payroll, d.annual_budget,
       ROUND(100.0*SUM(e.salary)/d.annual_budget,2) AS payroll_pct
FROM EmployeeRaw e JOIN DepartmentBudget d ON e.dept=d.dept
WHERE e.status='ACTIVE'
GROUP BY e.dept, d.annual_budget HAVING COUNT(DISTINCT e.eid)>0;
```

```
SELECT * FROM DeptAggregate ORDER BY payroll_pct DESC;
```

Output:

DE PT	EMP_CO UNT	AVG_SAL ARY	TOTAL_PAY ROLL	ANNUAL_BU DGET	PAYROLL_ PCT
CSE	2	88500	177000	5000000	3.54
ECE	1	78000	78000	4500000	1.73

Solution: Grouped view: department-level KPIs for budgeting and HR analytics.

Q106 Updatable View — WITH CHECK OPTION

Query:

```
CREATE VIEW ActiveEmployees AS
SELECT eid, ename, dept, salary
FROM EmployeeRaw WHERE status='ACTIVE'
WITH CHECK OPTION;
```

```
UPDATE ActiveEmployees SET salary=95000 WHERE eid=302;
SELECT eid, ename, salary FROM ActiveEmployees ORDER BY eid;
```

Output:

EID	ENAME	SALARY
301	Arjun	88000
302	Bhavna	95000
303	Chirag	78000

Solution: WITH CHECK OPTION: UPDATE succeeds if result satisfies WHERE predicate.

Q107 Complex Trigger — Cascade Audit and Auto-Calculation

Query:

```
CREATE TRIGGER trg_salary_impact
AFTER UPDATE ON EmployeeRaw
FOR EACH ROW WHEN (NEW.salary <> OLD.salary)
BEGIN
    INSERT INTO AuditLog(eid, action, old_value, new_value, audit_ts)
    VALUES (NEW.eid, 'SALARY_CHANGE', CAST(OLD.salary AS VARCHAR),
            CAST(NEW.salary AS VARCHAR), CURRENT_TIMESTAMP);
    UPDATE DepartmentBudget
    SET ytd_spent = ytd_spent + (NEW.salary - OLD.salary)
    WHERE dept=NEW.dept;
    UPDATE EmployeeRaw SET status='PROMOTED'
    WHERE salary > (SELECT AVG(salary)*1.5 FROM EmployeeRaw WHERE
    dept=NEW.dept);
END;
```

Output:

RESULT
Multi-action trigger: audit + budget adjustment + promotion logic executed

Solution: Complex trigger cascade: salary change triggers audit, budget adjustment, and promotion logic.

Q108 Materialized View — Incremental Refresh Strategy

Query:

```
CREATE MATERIALIZED VIEW mv_dept_performance AS
SELECT e.dept, COUNT(DISTINCT e.eid) AS headcount,
       ROUND(AVG(e.salary),0) AS mean_salary,
       ROUND(STDDEV_POP(e.salary),0) AS salary_stddev,
       MAX(e.salary) AS max_salary, MIN(e.salary) AS min_salary,
       SUM(e.salary) AS dept_payroll
FROM EmployeeRaw e WHERE e.status='ACTIVE'
GROUP BY e.dept;

REFRESH MATERIALIZED VIEW mv_dept_performance;
SELECT * FROM mv_dept_performance ORDER BY dept_payroll DESC;
```

Output:

DEPT	HEADCOUNT	MEAN_SALARY	SALARY_STDDEV	MAX_SALARY	MIN_SALARY	DEPT_PAYROLL
CSE	2	88500	3500	92000	85000	177000
ECE	1	78000	0	78000	78000	78000

Solution: Materialized view with REFRESH: periodic pre-computation of expensive aggregations.

Q109 Recursive CTE — Hierarchical Department Structure**Query:**

```

WITH RECURSIVE dept_hierarchy AS (
  SELECT dept, annual_budget, ytd_spent, 1 AS level
  FROM DepartmentBudget WHERE dept IN ('CSE','ECE')
  UNION ALL
  SELECT 'Finance', 3000000.00, 500000.00, 2
  FROM dept_hierarchy WHERE level < 2
)
SELECT dept, annual_budget, ytd_spent,
       ROUND(100.0*ytd_spent/annual_budget,1) AS budget_utilization
FROM dept_hierarchy
ORDER BY level, budget_utilization DESC;

```

Output:

DEPT	ANNUAL_BUDGET	YTD_SPENT	BUDGET_UTILIZATION
CSE	5000000	1800000	36
ECE	4500000	900000	20
Finance	3000000	500000	16.7

Solution: Recursive CTE for hierarchical queries; competitive exam advanced SQL feature.

Q110 Temporal Trigger — Window Function Audit Trail**Query:**

```

CREATE TRIGGER trg_temporal_audit
AFTER UPDATE ON EmployeeRaw FOR EACH ROW
WHEN (NEW.salary <> OLD.salary OR NEW.status <> OLD.status)
BEGIN

```

```

INSERT INTO AuditLog(eid, action, old_value, new_value, audit_ts)
VALUES (NEW.eid,
CASE WHEN NEW.salary<>OLD.salary
THEN 'SALARY_' || OLD.salary || '_TO_' || NEW.salary
ELSE 'STATUS_' || OLD.status || '_TO_' || NEW.status END,
CAST(OLD.salary AS VARCHAR), CAST(NEW.salary AS VARCHAR),
CURRENT_TIMESTAMP);
END;

SELECT eid, action,
ROW_NUMBER() OVER (PARTITION BY eid ORDER BY audit_ts DESC)
AS change_seq
FROM AuditLog WHERE eid=302 ORDER BY change_seq;

```

Output:

EID	ACTION	CHANGE_SEQ
302	SALARY_82000_TO_95000	1
302	HIRE_EVENT	2

Solution: Temporal audit triggers with window-function sequencing for historical analysis.

Q111 Constraint-Based Trigger — Budget Overrun Prevention

Query:

```

CREATE TRIGGER trg_budget_integrity
BEFORE UPDATE ON EmployeeRaw FOR EACH ROW
WHEN (NEW.salary <> OLD.salary)
BEGIN
IF (SELECT ytd_spent FROM DepartmentBudget WHERE dept=NEW.dept)
+ (NEW.salary - OLD.salary)
> (SELECT annual_budget FROM DepartmentBudget WHERE
dept=NEW.dept)
THEN
RAISE EXCEPTION 'Budget overrun: salary increase exceeds annual
budget';
END IF;
UPDATE DepartmentBudget
SET ytd_spent=ytd_spent+(NEW.salary-OLD.salary) WHERE
dept=NEW.dept;
END;

```

Output:

RESULT
Budget-aware trigger: subquery validates salary change against remaining budget

Solution: Complex constraint trigger: subquery validation before update; prevents data inconsistency.

Case Study 9: Query Processing, Execution Plans, and Performance Tuning

Units V

Setup — Schema & Data

SID	SNAME	GPA	DEPT_ID
1001	Alice	3.8	10
1002	Bob	3.5	10
1003	Carol	3.9	20
1004	David	3.2	10

COURSE_ID	CNAME	INSTRUCTOR_ID	CAPACITY	ENROLLED
2001	Data Structures	5001	60	45
2002	Algorithms I	5001	60	52
2003	Databases	5002	80	70

Q112 Index Scan vs Table Scan Decision

Query:

```
EXPLAIN SELECT sid, sname FROM Student WHERE sid=1003;
EXPLAIN SELECT sid, sname FROM Student WHERE dept_id=10;
```

Output:

QUERY	PLAN
WHERE sid=1003 (PK)	Index seek on PK (sid)
WHERE dept_id=10 (FK)	Seq scan vs Index scan (depends on selectivity)

Solution: Optimizer: PK seeks use index; non-key filters may use full scan if selectivity is high.

Q113 Join Execution — Nested Loop vs Hash Join

Query:

```
EXPLAIN ANALYZE
SELECT s.sname, c.cname
```

```
FROM Student s
JOIN StudentEnroll se ON s.sid=se.sid
JOIN CourseOffering c ON se.course_id=c.course_id;
```

Output:

JOIN_TYPE	EST_COST	ACTUAL_COST
Nested Loop Join	180 ms	175 ms
Hash Join (hash table fits memory)	120 ms	118 ms

Solution: Optimizer chooses hash join for large outer relation; nested loop for small inner.

Q114 Aggregate Pushdown in Execution Plan**Query:**

```
EXPLAIN
SELECT co.cname, COUNT(se.sid)
FROM CourseOffering co
LEFT JOIN StudentEnroll se ON co.course_id=se.course_id
GROUP BY co.course_id, co.cname;
```

Output:

PLAN_STEP
1. Sequential scan CourseOffering (outer table)
2. Index scan StudentEnroll filtered by course_id
3. Hash aggregate COUNT per course

Solution: Aggregate pushed after join; early filtering reduces intermediate result size.

Q115 Predicate Pushdown Optimization**Query:**

```
EXPLAIN
SELECT s.sname, c.cname
FROM (SELECT * FROM Student WHERE gpa>3.7) s
JOIN StudentEnroll se ON s.sid=se.sid
JOIN CourseOffering c ON se.course_id=c.course_id;
-- Optimizer pushes 'gpa>3.7' into base Student scan
```


Output:

OPTIMIZED_PLAN
Filter 'gpa>3.7' applied at Student table scan (predicate pushdown)
Reduces rows flowing to join by filtering early

Solution: Predicate pushdown: move WHERE conditions to earliest table scan; reduce join cardinality.

Q116 Column Projection Pruning**Query:**

```
SELECT s.sname, se.course_id
FROM Student s JOIN StudentEnroll se ON s.sid=se.sid;
-- Optimizer eliminates unused columns: gpa, dept_id not in SELECT
```

Output:

PRUNED_COLUMNS
gpa, dept_id removed from Student scan
Only sname and sid projected (sid needed for join)

Solution: Column pruning: select only needed attributes; reduces I/O and memory usage.

Q117 Index Selection — Cost Model Comparison**Query:**

```
EXPLAIN
SELECT * FROM StudentEnroll
WHERE course_id=2001 AND enroll_date>'2026-01-15';
-- Comparison: idx_course_id vs idx_composite(course_id,enroll_date)
```

Output:

INDEX_CHOICE	EST_COST	JUSTIFICATION
idx_course_id	250 ms	Filter by course_id; apply date filter on results
idx_composite (course_id,enroll_date)	80 ms	Both predicates in index; WINNER

Solution: Cost-based selection: composite index better for multi-column equality+range.

Q118 Parallelization — GROUP BY Partitioning**Query:**

```
EXPLAIN
SELECT dept_id, COUNT(DISTINCT sid)
FROM Student GROUP BY dept_id;
-- Potential parallelization: partition Student by dept_id
```

Output:

PARALLEL_PLAN
Partition Student by dept_id across 4 CPU cores
Parallel hash aggregate on each partition
Merge aggregate results (simple SUM of counts)

Solution: Parallel execution: partition large table; distribute GROUP BY; merge partial results.

Q119 Materialized Intermediate Result — Selectivity Estimation**Query:**

```
EXPLAIN
SELECT s.sname, COUNT(DISTINCT se.course_id)
FROM Student s LEFT JOIN StudentEnroll se ON s.sid=se.sid
WHERE s.gpa>3.7
GROUP BY s.sid, s.sname;
-- Decision: materialize filtered Student (small result) or keep as stream?
```

Output:

MATERIALIZATION_DECISION
Filtered Student: 2 rows (Alice gpa=3.80, Carol gpa=3.90)
Materialization beneficial: 2-row temp table vs re-scan Student multiple times
Strategy: materialize filter result, then hash join to StudentEnroll

Solution: Materialization: trade memory for I/O; beneficial when result cardinality small.

Q120 Statistics-Based Optimization — Histogram Cardinality**Query:**

```
SELECT 'Histogram on dept_id: dept_id=10 (3 students), dept_id=20 (1
student)' AS stats,
'Query: WHERE dept_id=10' AS predicate,
```

'Optimizer estimate: 3 rows (accurate via histogram)' AS accuracy;

Output:

STATS	PREDICATE	ACCURACY
dept_id=10:3 students, dept_id=20:1 student	WHERE dept_id=10	Estimate: 3 rows (accurate via histogram)

Solution: Statistics-based cardinality estimation: histograms guide index selection decisions.

Q121 Cost-Based Optimization — Alternative Plan Comparison

Query:

```
EXPLAIN(ANALYZE, BUFFERS, FORMAT JSON)
SELECT s.sname, se.course_id, c.cname
FROM Student s
JOIN StudentEnroll se ON s.sid=se.sid
JOIN CourseOffering c ON se.course_id=c.course_id
WHERE s.gpa>3.6
ORDER BY s.sname;
-- Plans: (1) Nested loop 3-way, (2) Hash join s+se then merge c, (3) Early
sort
```

Output:

PLAN_OPTION	COST_MS	MEMORY_MB	RECOMMENDATION
Plan 1: Nested Loop	450	5	Slow for large join result
Plan 2: Hash Join	180	20	Better: hash on (sid,course_id); then sort
Plan 3: Early Sort	220	8	Compromise: sort Student early; stream into join

Solution: Comparative cost: hash join $O(n)$ memory but fast; sort $O(n \log n)$ time; nested $O(n*m)$ interactions.

Case Study 10: Distributed Databases, Replication, and Consistency Models

Units V Advanced

Setup — Schema & Data

NODE_ID	NODE_NAME	LOCATION	IS_LEADER	SYNC_LAG_MS
1	Primary-NYC	NewYork	Y	0
2	Replica-EU	Frankfurt	N	150
3	Replica-AP	Singapore	N	300

MODEL_ID	MODEL_NAME	DESCRIPTION
1	Strong Consistency	All replicas must ACK before commit
2	Eventual Consistency	Commit then async replicate; eventual state match
3	Causal Consistency	Causally related ops maintain order; unrelated may be concurrent

Q122 Replication Lag and Consistency Implications

Query:

```
SELECT rn.node_name, tl.data_version, tl.committed_ts, rn.sync_lag_ms,  
       DATEDIFF(MILLISECOND, tl.committed_ts, CURRENT_TIMESTAMP) AS  
       age_ms  
FROM ReplicaNode rn LEFT JOIN TransactionLog tl ON  
     rn.node_id=tl.node_id  
ORDER BY age_ms DESC;
```

Output:

NODE_NAME	DATA_VERSION	COMMITTED_TS	SYNC_LAG_MS	AGE_MS
Replica-AP	2	2026-02-15 10:32:15	300	3600000
Replica-EU	2	2026-02-15 10:31:30	150	3600000
Primary-NYC	2	2026-02-15 10:31:00	0	3600000

Solution: Lag indicates eventual consistency: AP replica 300ms behind primary.

Q123 Strong vs Eventual Consistency Tradeoffs

Query:

```
SELECT COUNT(CASE WHEN replication_status='REPLICATED' THEN 1
END) AS replicated_txns,
       COUNT(CASE WHEN replication_status='IN_PROGRESS' THEN 1 END)
AS pending_txns
FROM TransactionLog
WHERE committed_ts > DATEADD(MINUTE,-5,CURRENT_TIMESTAMP);
```

Output:

REPLICATED_TXNS	PENDING_TXNS	STRONG_DESC	EVENTUAL_DESC
3	1	Wait all ACK: slower, consistent	Commit local, async replicate: faster

Solution: Strong consistency: wait for majority; eventual allows fast commit with consistency lag.

Q124 Split-Brain Detection — Both Nodes Claim Leadership

Query:

```
SELECT COUNT(CASE WHEN is_leader='Y' THEN 1 END) AS leader_count
FROM ReplicaNode;
SELECT CASE WHEN (SELECT COUNT(CASE WHEN is_leader='Y' THEN 1
END) FROM ReplicaNode) > 1
THEN 'SPLIT_BRAIN_DETECTED' ELSE 'HEALTHY' END AS
cluster_status;
```

Output:

LEADER_COUNT	CLUSTER_STATUS
1	HEALTHY

Solution: Single leader enforced: prevents split-brain where conflicting writes occur.

Q125 Quorum Monitoring — Majority ACK for Commit Safety

Query:

```
SELECT COUNT(*) AS total_nodes, CEILING(COUNT(*)/2.0)+1 AS
quorum_size FROM ReplicaNode;
```

```

SELECT SUM(CASE WHEN replication_status='REPLICATED' THEN 1 ELSE
0 END) AS acked
FROM TransactionLog WHERE txn_id=1002;
SELECT CASE WHEN acked >= quorum_size THEN 'QUORUM_REACHED'
ELSE 'NO_QUORUM' END AS consensus;

```

Output:

TOTAL_NODES	QUORUM_SIZE	ACKED	CONSENSUS
3	2	2	QUORUM_REACHED

Solution: Quorum commit: majority ACK guarantees durability and partition tolerance.

Q126 Vector Clocks — Causality and Ordering**Query:**

```

SELECT 'T1 on Node1: VC=[1,0,0]' AS t1,
      'T2 on Node2 after T1: VC=[1,1,0]' AS t2,
      'T3 on Node3 after T1,T2: VC=[1,1,1]' AS t3,
      'T1->T2 causality ensured by VC comparison' AS causality;

```

Output:

T1	T2	T3	CAUSALITY
VC=[1,0,0]	VC=[1,1,0]	VC=[1,1,1]	T1→T2 causality via VC comparison

Solution: Vector clocks: track causality; ensure causal consistency across replicas.

Q127 Raft Consensus — Leader Election and Log Replication**Query:**

```

SELECT 'Phases: (1) Leader election via heartbeat timeout,
      (2) Log replication via follower ACK,
      (3) Commit when majority ACK' AS raft_phases,
      'Log entry: term=5, index=100, data=balance_update' AS example;

```

Output:

RAFT_PHASES	EXAMPLE	SAFETY
(1) Election, (2) Replication, (3) Commit on majority	term=5, idx=100, data=balance_update	Safe: committed only if replicated to majority

Solution: Raft consensus: linearizable, fault-tolerant; standard in distributed DB systems.

Q128 Two-Phase Commit (2PC) — Protocol and Failure Handling

Query:

```
SELECT 'Phase 1 PREPARE: coordinator asks replicas "Can you commit?"'
AS phase1,
      'Phase 2 COMMIT: if all YES, send COMMIT; else ABORT' AS phase2,
      'Failure: coordinator fails after Phase 1 => replicas blocked' AS blocker;
```

Output:

PHASE1	PHASE2	BLOCKER
Coordinator: Can you commit?	All YES: COMMIT; else ABORT	Coordinator failure: orphaned transaction blocked

Solution: 2PC atomicity across distributed sites; vulnerable to coordinator failure (blocking).

Q129 CAP Theorem — Tradeoff Analysis for Geo-Distributed DB

Query:

```
SELECT 'Partition: Primary-NYC <-> Replica-EU' AS scenario,
      'CP: strong+partition tolerant (available only at primary)' AS cp_choice,
      'AP: available+partition tolerant (eventual consistency at EU)' AS
ap_choice,
      'Geo-replicated DBs typically choose AP' AS decision;
```

Output:

SCENARIO	CP_CHOICE	AP_CHOICE	DECISION
Network partition NYC-EU	Strong+tolerant; EU unavailable	Available; EU serves stale	Geo DBs: choose AP (eventual)

Solution: CAP impossible in partition; choose 2: CP (strong+partition tolerant) or AP (available+partition tolerant).

Q130 CRDT — Conflict-Free Replicated Data Type

Query:

```
SELECT 'CRDT Counter (increment-only)' AS type,
      'Replica-A: increment by 2 (state=2), Replica-B: increment by 3
(state=3)' AS ops,
      'Merge: state = 5 (2+3); order-independent' AS merge,
```

'Properties: commutative, associative, idempotent' AS properties;

Output:

TYPE	OPS	MERGE	PROPERTIES
Counter (increment-only)	A: +2 (2), B: +3 (3)	Merge: 5 (order-independent)	Commutative, associative, idempotent

Solution: CRDTs: automatically merge without coordination; convergence property guaranteed.

Q131 Eventual Consistency — Read-Repair and Anti-Entropy

Query:

SELECT 'Read-Repair: client reads v2 from A but v1 from B; write v2 back to B' AS read_repair,

'Anti-Entropy: background Merkle tree comparison; propagate divergent data' AS anti_entropy,

'Hinted Handoff: temp-store write if target down; retry on recovery' AS handoff;

Output:

READ_REPAIR	ANTI_ENTROPY	HANDOFF
Client detects stale; writes v2 back to B	Background Merkle comparison; divergent sync	Temp-store write; retry when target recovers

Solution: Consistency repair: read-repair on query, anti-entropy background; converge to consistent state.

Case Study 11: Comprehensive Integration — Schema Design, Normalization, Optimization

All Units

Setup — Schema & Data

```
CREATE TABLE University(uid INT PRIMARY KEY, uname VARCHAR(100),
location VARCHAR(50), founded_year INT);
CREATE TABLE Department(dept_id INT PRIMARY KEY, dname
VARCHAR(80), uid INT,
annual_budget DECIMAL(12,2), FOREIGN KEY(uid) REFERENCES
University(uid));
CREATE TABLE Faculty(fid INT PRIMARY KEY, fname VARCHAR(40),
dept_id INT,
salary DECIMAL(10,2), qualification VARCHAR(50), FOREIGN
KEY(dept_id) REFERENCES Department(dept_id));
CREATE TABLE Course(cid INT PRIMARY KEY, cname VARCHAR(100),
credit_hours INT,
dept_id INT, FOREIGN KEY(dept_id) REFERENCES Department(dept_id));
CREATE TABLE Section(sec_id INT PRIMARY KEY, cid INT, fid INT,
semester VARCHAR(10),
max_capacity INT, FOREIGN KEY(cid) REFERENCES Course(cid),
FOREIGN KEY(fid) REFERENCES Faculty(fid));
CREATE TABLE StudentMaster(sid INT PRIMARY KEY, sname
VARCHAR(80), dept_id INT,
admission_date DATE, gpa DECIMAL(3,2), FOREIGN KEY(dept_id)
REFERENCES Department(dept_id));
CREATE TABLE Enrollment(eid INT PRIMARY KEY, sid INT, sec_id INT,
grade CHAR(2),
FOREIGN KEY(sid) REFERENCES StudentMaster(sid), FOREIGN
KEY(sec_id) REFERENCES Section(sec_id));
```

UID	UNAME	LOCATION	FOUNDED_YEAR
100	Tech University	Silicon Valley	1985

DEPT_ID	DNAME	UID	ANNUAL_BUDGET
10	Computer Science	100	8000000
20	Electrical Engineering	100	7000000

FID	FNAME	DEPT_ID	SALARY	QUALIFICATION
501	Dr. Rao	10	120000	PhD
502	Dr. Patel	10	110000	PhD
503	Dr. Verma	20	115000	PhD

SID	SNAME	DEPT_ID	ADMISSION_DATE	GPA
1001	Alice	10	2023-08-15	3.85
1002	Bob	10	2023-08-15	3.65
1003	Carol	20	2023-08-20	3.75

Q132 ER Schema Review — Entities, Relationships, Cardinality

Query:

SELECT 'Entities: University, Department, Faculty, Course, Section, Student'
AS entities,
 'University 1-N Department, Department 1-N Faculty/Course/Student'
AS cardinality;

Output:

ENTITIES	CARDINALITY
University, Department, Faculty, Course, Section, Student	1-N: Univ→Dept, Dept→Faculty/Course/Student, Faculty→Section, Course→Section

Solution: ER design normalized: each entity separate; junction table Section for Faculty-Course assignment.

Q133 Verify 1NF — Atomic Attributes, No Repeating Groups

Query:

SELECT 'Faculty(fid,fname,dept_id,salary,qualification): all atomic' AS
check1,
 'Enrollment(eid,sid,sec_id,grade): atomic, no multi-valued grade' AS
check2;

Output:

CHECK1	CHECK2
Faculty: all atomic, no repeating groups	Enrollment: atomic (no multi-valued grade)

Solution: 1NF satisfied: all attributes single-valued; no composite or multi-valued domains.

Q134 Verify 2NF — No Partial Dependency on Composite Key**Query:**

```
SELECT 'Primary key Enrollment=(eid) is single, not composite' AS pk,
       'No partial dependencies possible in single-key relations' AS result;
```

Output:

PK	RESULT
Enrollment PK=(eid): single key	No partial dependencies possible

Solution: 2NF satisfied: single-key relations automatically 2NF; no partial dependencies.

Q135 Verify 3NF — No Transitive Dependencies**Query:**

```
SELECT 'Faculty: fid->salary directly (no transitive chain)' AS check_faculty,
       'Student: sid->gpa is direct dependency' AS check_student;
```

Output:

CHECK_FACULTY	CHECK_STUDENT
fid→salary, qualification: direct (no transitive chain)	sid→gpa: direct dependency, no transitive

Solution: 3NF satisfied: all non-prime attributes directly dependent on primary key.

Q136 Complex Multi-Table Query — CTE + Window + Aggregation**Query:**

```
WITH dept_analytics AS (
  SELECT d.dept_id, d.dname,
         COUNT(DISTINCT f.fid) AS faculty_count,
         COUNT(DISTINCT c.cid) AS course_count,
```

```

ROUND(AVG(f.salary),0) AS avg_faculty_salary,
RANK() OVER (ORDER BY COUNT(DISTINCT c.cid) DESC) AS
course_vol_rank
FROM Department d
LEFT JOIN Faculty f ON d.dept_id=f.dept_id
LEFT JOIN Course c ON d.dept_id=c.dept_id
GROUP BY d.dept_id, d.dname
)
SELECT dept_id, dname, faculty_count, course_count,
avg_faculty_salary, course_vol_rank
FROM dept_analytics ORDER BY course_vol_rank;

```

Output:

DEPT_ID	DNAME	FACULTY_COUNT	COURSE_COUNT	AVG_SALARY	RANK
10	Computer Science	2	2	115000	1
20	Electrical Engineering	1	1	115000	2

Solution: Complex query: CTE for analytics, window ranking, multi-table aggregation; competitive standard.

Q137 Index Strategy — Query Speed vs Update Overhead**Query:**

```

SELECT 'Frequent lookup by sid: student enrollment history' AS pattern1,
'Range search on gpa: Dean list identification' AS pattern2,
'Decision: CREATE INDEX idx_student_gpa ON StudentMaster(gpa);
CREATE INDEX idx_enrollment_sid ON Enrollment(sid)' AS
indexes;

```

Output:

PATTERN1	PATTERN2	INDEXES
Frequent lookup by sid	Range search on gpa	idx_student_gpa + idx_enrollment_sid

Solution: Index selection: gpa for range queries; sid for join efficiency; balance with INSERT/UPDATE cost.

Q138 Schema Evolution — Add Column with Backward Compatibility**Query:**

```
ALTER TABLE Faculty ADD COLUMN research_interest VARCHAR(200)
DEFAULT 'Not Specified';
SELECT 'Old queries still work (column ignored)' AS backward_compat,
       'New queries access research_interest' AS new_feature;
```

Output:

BACKWARD_COMPAT	NEW_FEATURE
Old queries still work (column ignored)	New queries can access research_interest

Solution: Schema evolution: add nullable/default column; maintains backward compatibility.

Q139 Temporal Query — Historical Student GPA Tracking**Query:**

```
CREATE TABLE StudentGradeHistory(
  sid INT, gpa DECIMAL(3,2),
  valid_from DATE, valid_to DATE,
  FOREIGN KEY(sid) REFERENCES StudentMaster(sid)
);

SELECT 'Find student GPA as of 2024-12-31' AS historical_query,
       'WHERE valid_from <= 2024-12-31 AND (valid_to IS NULL OR valid_to
       >= 2024-12-31)' AS filter;
```

Output:

HISTORICAL_QUERY	TEMPORAL_FILTER
Find student GPA as of 2024-12-31	valid_from <= 2024-12-31 AND (valid_to IS NULL OR valid_to >= 2024-12-31)

Solution: Temporal database design: validity dates enable historical queries and audit trails

Q140 Optimistic Locking — Concurrent Enrollment Modification**Query:**

```
SELECT 'Version column: Enrollment.version=1' AS approach,
       'UPDATE WHERE eid=5001 AND version=1; SET grade=A, version=2' AS
update,
       'If WHERE fails (version changed): retry or abort' AS conflict_resolution;
```

Output:

APPROACH	UPDATE	CONFLICT_RESOLUTION
Version column: Enrollment.version=1	WHERE eid=5001 AND version=1; SET version=2	WHERE fails: retry or abort

Solution: Optimistic locking: version numbers detect conflicts; retry on mismatch.